

Technology agnostic design systems

Alexsi Koivu

School of Science

Thesis submitted for examination for the degree of Master of Science in Technology.

Espoo 01.09.2022

Thesis supervisor:

Petri Vuorimaa

Thesis advisor:

Piia Alavesä



Author Aleksi Koivu

Title Technology Agnostic Design Systems

Major Computer Science**Code of major** SCI3042

Degree programme Master's Programme in Computer, Communication and Information Sciences

Supervisor Prof. Petri Vuorimaa

Advisor M.Sc. Piia Alavesä

Date 01.09.2022**Number of pages** 57 + 14**Language** English

Abstract

Design systems are becoming vital parts of facilitating the creation of digital products in large companies. Metso Outotec has a design system that has a React specific component library. For a design system to be compatible with any digital product, it must not be bound to one technology, i.e., it must be technology agnostic. This thesis explores the value of a design system, and the options of creating a technology agnostic component library. The results will cover user survey data, key performance indicators, and a practical minimum viable product implementation of such a component library for Metso Outotec, called Modes UI.

Keywords technology agnostic, design system, framework, front-end, component library, component, React, Lit, Web Components

Tekijä Aleksi Koivu

Työn nimi Teknologia-agnostiset suunnittelujärjestelmät

Pääaine Computer Science**Pääaineen koodi** SCI3042

Koulutusohjelma Master's Programme in Computer, Communication and Information Sciences

Työn valvoja Prof. Petri Vuorimaa

Työn ohjaaja FM Piia Alavesa

Päivämäärä 01.09.2022**Sivumäärä** 57 + 14**Kieli** Englanti

Tiivistelmä

Suunnittelujärjestelmät ovat tulossa tärkeäksi osaksi digitaalisten tuotteiden kehittämisen helpottamista suurissa yrityksissä. Metso Outotecilla on design systeemi, jonka komponenttikirjasto perustuu Reactiin. Jotta design systeemi voisi olla yhteensopiva kaikkien digitaalisten tuotteiden kanssa, se ei voi olla sidottu yhteen teknologiaan. Toisin sanoen, sen pitäisi olla teknologia-agnostinen. Tämä diplomityö tutkii design systeemien tuomaa arvoa, ja mitä vaihtoehtoja teknologia-agnostisen komponenttikirjaston luomiseen on olemassa. Työn tulokset käsittelevät käyttäjäkyselyn dataa, tärkeitä performanssimetrikoita, sekä Metso Outotecille luotua uutta komponenttikirjaston pienimmän julkaisukelpoisuuden tuotetta nimeltä Modes UI.

Avainsanat teknologia, agnostinen, design, järjestelmä, työkalu, front-end, komponentti, komponenttikirjasto, React, Lit, Web Components

PREFACE

I want to thank my supervisor Prof. Petri Vuorimaa and my advisor M. Sc. Piia Alavesa for their support and quick and knowledgeable feedback. Both have been immensely helpful and always available. A big thank you to Janne Lampinen, Jussi Järvinen, and Metso Outotec as a whole for giving me the opportunity to do this thesis for them and providing me with the necessary time and environment. I would also like to thank my family for their continuous support and feedback, it has been invaluable.

Espoo, 01.09.2022

Aleksi Koivu

ABBREVIATIONS & TERMINOLOGY

Abbreviations

MODES	Metso Outotec Design System
DSUI	Design System User Interface, i.e. React component library
UI	User Interface
UX	User Experience
MVP	Minimum Viable Product
POC	Proof Of Concept
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
TS	TypeScript
DOM	Document Object Model
API	Application Programming Interface
NPM	Node Package Manager

Terminology

Author

an author is a person who works on creating and managing the component library.

Component library

the collection of web components that have been implemented based on the designs in the UI Kit. Individual components are packaged into a library to ease distribution to users.

DevOps

A combination of classical development (Dev) and operational (Ops) tasks. As a result, teams and organizations that use DevOps are able to deliver and improve their products at a higher velocity than those using standard development and infrastructure management processes [47].

End-user

an end-user is someone who uses products built using the design system, but is not a direct user of the design system itself.

Product Team

a product team in this context usually consists of a Product Owner (PO), designers, and developers. Product teams are responsible for creating and maintaining their own products, and they are the primary users of a company-internal design system.

UI Kit

UI kit stands for “User Interface kit”. A UI Kit in this context refers to the collection of components designs and other atomic elements that designers use to build new pages and templates.

User

a user in this thesis refers to someone who directly utilizes the design system.

LIST OF FIGURES

2.1	What is a Design System?	5
2.2	The tools used in Modes and their structure within the design system. . . .	6
2.3	Design system principles inside Metso Outotec.	7
2.4	Google's Material Design System	7
2.5	Google search trends for the term "design system" (past 10 years).	8
2.6	Benefits of reusable components.	9
2.7	Modes UI (see Section 5.1) tab composition.	9
2.8	Some of the Metso Outotec brand tokens provided by the design system. . .	12
2.9	Atomic design	15
2.10	Color themes	16
3.1	An example of a technology agnostic design system.	17
3.2	Web Components can be used across different frameworks.	19
3.3	The technologies behind Web Components.	20
3.4	The shadow tree encapsulates styling and functionality.	22
3.5	A button with different slots.	24
4.1	CI/CD Pipeline	31
4.2	LitElement class inheritance.	36
5.1	Modes UI documentation website	43
5.2	Differences between date time range implementations.	46
5.3	Source code is provided for both general usage and React specific usage. . .	47
5.4	Modes UI component documentation example.	48
5.5	Vue supports almost all Web Component features.	55

LIST OF TABLES

1.1	Thesis chapter structure.	4
3.1	Legend for Fig. 3.4.	22
3.2	Web Component browser support	25
3.3	A comparison between the most popular Web Component libraries.	27
5.1	Modes UI "dependencies" table.	44
5.2	Modes UI "devDependencies" table.	45
5.3	Component differences between DSUI and Modes UI.	46
5.4	Modes is regularly accessed from all over the globe.	54

LIST OF CODE

3.1	An autonomous custom element.	21
3.2	A customized built-in element.	21
3.3	Slotting content in HTML templates.	23
3.4	Slotted button example from Modes UI.	24
4.1	A simple unit test.	30
4.2	Hello World component in Lit	34
4.3	HTML to import and render the component.	35
4.4	Additional TypeScript typing for a Lit component.	36
4.5	An external module that exports a tagged template literal.	37
4.6	Importing external styles in a Lit component.	37
4.7	Dynamic class assignment using classMap	38
4.8	Using custom properties to define component style.	38
4.9	Declaring an internal state property for a component in Lit.	39
4.10	Assigning an initial value to a property through attributes.	40
4.11	Assigning an initial value directly to a property in React.	40
4.12	Different configurations for Lit attribute handling.	40
4.13	Reflecting property values to attributes.	41
5.1	Dynamically creating React wrapped components.	50
5.2	Event handling in React using custom events.	51
5.3	Form submission in React using Modes UI components.	52

CONTENTS

Abstract	ii
Abstract (Finnish)	iii
Preface	iv
Abbreviations & terminology	v
List of Figures	vii
List of Tables	viii
List of Code	
1 Introduction	1
1.1 Background	1
1.1.1 Case study: Metso Outotec	1
1.1.2 Problem statement	2
1.1.3 Motivation	3
1.2 Scope and research questions	3
1.2.1 Thesis structure	4
2 Design systems	5
2.1 The importance and value of design systems	6
2.1.1 Value in development	8
2.1.2 Intangible benefits	11
2.2 The challenges of design systems	12
2.2.1 Justifying initial investment	12
2.2.2 Adoption	13
2.2.3 Maintenance	15
2.2.4 Design systems in industrial context	15
3 Technology agnosticism	17
3.1 Introduction to technology agnosticism	17

3.2	Web Components	19
3.2.1	Custom Elements	20
3.2.2	Shadow DOM	22
3.2.3	HTML Templates & Slots	23
3.3	Support, migration & adoption	24
3.3.1	Support	24
3.3.2	Migration & adoption	25
3.4	Choosing a Web Component library	26
3.4.1	Web Component compilers	27
3.4.2	Lit	28
4	Implementation	29
4.1	Component library management	29
4.1.1	Testing	30
4.1.2	Dependencies	32
4.1.3	Documentation & distribution	33
4.2	Creating components with Lit	34
4.2.1	Defining a component	35
4.2.2	Scoped styles	36
4.2.3	Reactive properties	39
4.2.4	Rendering & templates	41
5	Results	42
5.1	Modes UI	42
5.1.1	Project dependencies	44
5.1.2	DSUI comparison	45
5.1.3	Providing React support	49
5.2	Users	52
5.2.1	Survey results	52
5.2.2	Key performance indicators	53
5.2.3	Case study: Intelligent platforms	55
6	Conclusion	56
	References	58

INTRODUCTION

1.1 Background

A **design system** is a set of guidelines for managing design at scale by avoiding redundancy and establishing a common language and aesthetic across several pages and channels [14]. From a developer's perspective, the most important part of a design system is a set of reusable components that can be used together to build user interfaces for applications, that are often web-based. These reusable components form a **component library** that adheres to the design and development principles that are established and documented by the design system.

As a web developer, an integral part of the work is choosing the correct tools for the project. This is often not an easy task, however, as developers need to consider further than just the immediate needs of the project. The important questions are *how* and for *how long* will the project be maintained? [13]

Especially in web development, there is an incredible amount of different tools, frameworks and libraries available to choose from. It can easily get confusing and create extra dependencies that may cause issues in the future. "Framework fatigue" has been a hot topic in web development due to this, and there is a need for a more holistic solution than combining multiple unrelated technologies and libraries.

Web components are a new browser standard aimed to alleviate these issues to an extent. Creating a component library from web components would make the design system more flexible and more accessible to more users. This would also ensure longevity for the implementation, as browser standards are guaranteed to be maintained for a long time. This thesis will explore the creation of a **technology agnostic** component library for Metso Outotec, using web components as the backbone of the implementation.

1.1.1 Case study: Metso Outotec

Metso Outotec is one of Finland's largest companies, that strives to become a "tier one" company. Metso Outotec was "born" in 2020, when two large companies in Metso and Outotec merged together. Metso Outotec now has more than 15 000 employees across over 50 different countries. These employees form hundreds of product teams, that serve thousands of customers worldwide. Many of these teams have digital products and web applications that could benefit from a design system. In August 2020, Metso Outotec began work on creating their own design system, that would improve branding and consistency in the company's online products. For this task, a design system team was created.

There was some previous work done on creating shareable components, but there was never a team dedicated to the creation and maintenance of a design system. Outotec already had a basic component library, which was then used as a base for the new design system

implementation work. After the merging, all of the components had to be re-branded to match the brand of Metso Outotec. Having a base component library saved a lot of time, as adding branding on existing functional components is less laborious than creating new ones from scratch.

This is a testament to the power of a design system, teams that were using the former Outotec component library were able to update to the correctly branded components by just updating their existing library to the newest release. This makes the transition to the new branding faster, and allows the product teams to focus on improving their product, instead of branding. A common design system is cost-effective long-term, as product teams can use the ready-made components instead of creating their own components from scratch.

In one and a half years, Metso Outotec has developed a fairly mature design system that has benefited over two dozen reference products and the documentation has hundreds of visitors. The design system provides users exhaustive documentation and the designs strive to improve the accessibility and user experience provided by the components. It is maintained by the **Metso Outotec design system (Modes)** team, and its user base is growing at a steady rate. It was initially created to be used by only a couple of product teams, but now it has been widely adopted by many different teams and products.

1.1.2 Problem statement

Currently, adoption of the **design system component library (DSUI)** requires the product teams to use React as their front-end framework. This has led to some teams not being able to use the ready-made components provided by the design system team, as they are not using React. This means teams can not utilize a significant part of the design system, resulting in extra work duplicating component implementations, and possibly a loss of consistency and brand identity.

Recently a team inside Metso Outotec had to invest a lot of resources into the recreation of many components, as they moved parts of their product to Salesforce, that does not allow the usage of React components. As Salesforce components were based on Lightning web components, the team had no prior experience on them, and chose to use a consulting firm for the component creation. This is expensive and may cause issues in the future when those components need to be changed. Multiple sets of component libraries are also bound to have inconsistencies across them, which harms the strength of the branding, and their functionality and re-usability is not guaranteed.

Another issue that has come up is the lifetime of a product. In a large, industrial company, there are hundreds of products with varying needs. Some products are built with the idea that they can, and likely will, exist for decades. In such a scenario, relying on a framework such as React can become an issue, as then the lifetime of the project is tied to that of the framework's.

The solution to these problems that the thesis will explore is creating a new version of the current component library that can be used in combination with any proprietary web technology (e.g., Angular, Vue and Flask). This is what a **technology agnostic** design system means; being compatible with any other web technologies that the teams may want to utilize. Creating a technology agnostic version of the component library will allow even

more current and future product teams to adopt the design system along with its component library.

1.1.3 Motivation

Improving the adoption rate of the design system benefits everyone involved. Product teams save time on implementation, and the design system team receives great feedback that they can use to further improve. Ideally these product teams can also contribute to the design system with new designs and implementations. Maintaining strong brand identity becomes difficult in such a large company as well, and a widely adopted design system can help a lot in achieving a consistent and efficient layout across different websites and products.

Web development tools and frameworks are constantly changing and evolving, and it can be hard to choose the correct framework for a new project. The current component library is based on React, and even though it recently surpassed jQuery as the the most popular framework [40], teams may want to migrate to another framework later, or simply start with a different one. Creating a **technology agnostic** component library gives product teams the freedom of choosing the framework that best suits their needs, while still being able to utilize the comprehensive component library.

1.1.3.1 Personal motivation

In my one year of working on the design system I have learned to appreciate its benefits, but I have noticed its adoption could be improved. Getting a better understanding of the technological architecture and structure inside a global corporation is something that I wish to achieve. So far I have come across many challenges concerning the current design system, and I hope to be able to alleviate or even get rid of some of these issues.

1.2 Scope and research questions

This thesis studies the benefits and obstacles of creating a technology agnostic design system in Metso Outotec. A general overview of design systems and the value they provide will be useful later on to estimate the value added by implementing a technology agnostic version. For the implementation part, research will first be carried out to find the best tool for the task. Then, a **Proof of Concept (POC)** implementation will be done for a sub-set of components to assess the difficulty of migrating all components to a technology agnostic version of the component library.

The objective of this thesis is to analyze the business value achieved by the creation of a technology agnostic design system and to create a comprehensive plan to accomplish this inside Metso Outotec. This thesis attempts to estimate the monetary and time cost to evaluate whether this undertaking is worthwhile. This will be done by analyzing the existing user structure inside the company and surveying how many future and current product teams could adopt the new system. This will then be compared to the current React-based library to find the value gained by the creation of the new library.

The thesis will also attempt to find solutions to other design system related issues that can emerge in a large company. A large company will have tens or hundreds of different teams,

each with varying needs. This makes creating and maintaining a common design system that can serve everyone difficult. Ways to improve communication will be discussed along with some Metso Outotec specific practices and their effects on work efficiency.

The focus of this thesis is on the implementation aspect of a design system. It **does not** thoroughly explore the underlying design principles and documentation, or their effects on the efficiency and value of a design system. However, design and documentation are a large part of a design system so they will be contextualized to an extent.

This thesis attempts to answer the following research questions:

1. How much concrete value do design systems provide?
2. What are the benefits of creating a technology agnostic design system?
3. How should a large company migrate to a technology agnostic design system?
4. How does the implementation of web components differ from React?

1.2.1 Thesis structure

Chapter	Content
Design systems	The second chapter will introduce the reader to design systems and explore the impact and value that a design system can bring to a company. It will also examine the challenges it may present in a large company, and how to possibly remedy those.
Technology agnosticism	The third chapter will be about explaining the underlying principles behind a technology agnostic design system. It will explain the concept of Web Components, and how they function and enable a technology agnostic component library.
Implementation	The fourth chapter will focus on the creation of a technology agnostic component library. It will go through the component library configuration and setup, and the component creation workflow.
Results	The fifth chapter will examine the results of creating a proof of concept component library for Metso Outotec's internal usage. The new component library will be compared to the existing React counter-part, weighing the pros and cons of each. Survey results and performance metrics will be examined to find the objective, observable differences between different options.
Conclusion	The final chapter will conclude the thesis with a summary of the findings and how a design system functions inside a large company, such as Metso Outotec.

Table 1.1: Thesis chapter structure.

DESIGN SYSTEMS

As alluded to in the introduction chapter, design systems provide guidelines and components that ease the development of new products. They function as a single source of truth, to ensure that all involved parties are on the same page when creating digital products and solutions. These building blocks are consistent in style and functionality, which enhances the User eXperience (UX) across multiple product teams that have adopted the design system. Ideally the users of the design system also contribute to the design system by collaboratively building new designs and components.

What is a Design System?		
A collection of ↓	Used by both ↓	Controlled with ↓
Foundations Basic building blocks Colors Typography Icons Spacing Grids Motion	Design Assets created and used in a design tool Design Files Libraries Styles	Guidelines Written usage, rules and recommendations Accessibility Examples Patterns Principles Usage Voice & Tone
Components Reusable UI elements made with foundations Buttons Cards Input fields Pagination Tabs Tables	Engineering Implementation in front-end development APIs Components Tokens	Governance Adoption, maintenance processes, analysis Communication Contribution Incentives Measurement Reviews Versions

Figure 2.1: What is a Design System? [17]

In practice, design systems consist of three main parts: design, implementation, and documentation. As the name implies, designs are a fundamental part of the design system, and they function as the basis for the implementation work. The implementation is usually a component library, that has been compiled for a specific framework (React in Metso Outotec's case). The documentation has guidelines on usage scenarios and best practices

in design and development. Comprehensive documentation is crucial, as it saves time by reducing the number of issues the users encounter due to incorrect usage. Exhaustive documentation also helps designers and developers who work on the design system. These three main parts need to be tightly integrated to create a suitable workflow for design system development, and to ease adoption for design system users.

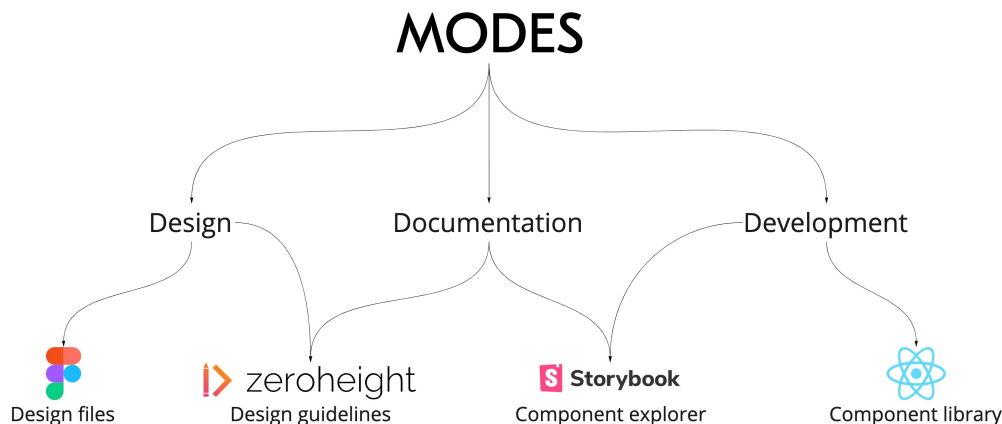


Figure 2.2: The tools used in Modes and their structure within the design system.

Various useful tools have been created to address the needs of a design system. The logos of some of the most well-known ones are in Fig. 2.2 above: Figma for design files, zeroheight for design guidelines, and Storybook for showing an interactive demo of the component library. However, most sophisticated tools such as these require a license to be utilized inside a corporation. Enterprise licensing can prove to be a monumental task, as fees can become surprisingly hefty, and reviewing and signing legal and terms agreements can take months. This poses a challenge in the initial phase of the creation of a design system: which tools should be licensed from third-party businesses, and which ones should be built as part of the system?

In this chapter, the value of design systems is examined from multiple different viewpoints and, where possible, its monetary value is roughly estimated to make the findings more palpable. Existing design systems will be used as benchmarks and examples of good practices. The challenges of building and maintaining a design system are also studied, to have a better understanding of what the major pain points in creating a design system are and how they should be remedied.

2.1 The importance and value of design systems

The ever-growing need for the creation of functional, adaptive, and versatile user interfaces (UIs) creates a massive amount of work for designers and developers. Many have begun adopting frontend toolkits, such as [Bootstrap](#), to shift some of the workload on to an existing solution. As of 2016, Bootstrap was the most popular repository on the industry standard code-sharing site GitHub, with over 77,000 stars and 30,000 forks [15]. The popularity of such toolkits and frameworks is a testament to how many designers and developers are looking for a solid foundation of building blocks to build their projects and products on.

The rising popularity of design systems is a fairly recent phenomenon, as most of the growth has occurred in the past five years (see Fig. 2.5). Partly due to this, academic research that assesses their value is sparse. We can, however, deduce from the increase in adoption that they are perceived as increasingly valuable. Their value is hard to quantify though, as e.g., the value of branding consistency and an unified user experience is not directly perceptible.

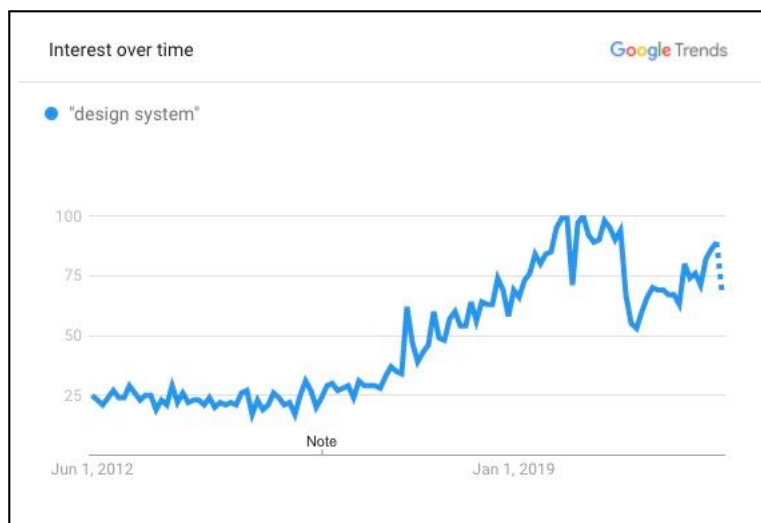


Figure 2.5: Google search trends for the term "design system" (past 10 years).

Having a proper design system generates value through time-savings for both developers and designers. It is difficult to quantify these time-savings, especially for designers. Clancy Slack, a data scientist at **Figma**, built an experiment to attempt to quantify the time-savings design systems can create for designers. To quantitatively measure a creative process, they created a scenario where participants (designers) had to complete two different tasks within the context of a bank account aggregator app. The experiment was controlled to eliminate bias, and the results showed that participants with an access to a design system completed their task **34%** faster than those without one. This increase in efficiency translates to designers contributing **50%** more in the same amount of time. In other words, a team with two designers and a design system can perform as well as a team with three designers without one (in number of tasks completed), based on this experiment. [39]

Design systems allow product teams to work better together and the teams are freed up to focus on what is important: solving customer problems and creating value for them [1]. And when the features of components or their functionality inevitably change, the change is automatically reflected across all products that use the design system.

2.1.1 Value in development

As shown above, designers can work more efficiently when they can take advantage of the guidelines and design files that a design system offers. This section will attempt to quantify the value that a design system provides for developers, and through which avenues most value is generated.

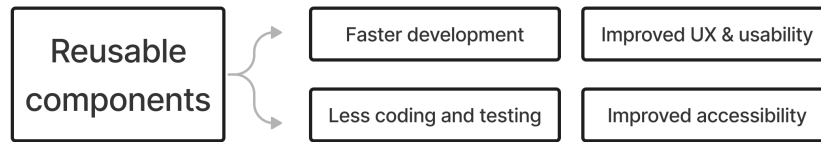


Figure 2.6: Benefits of reusable components.

2.1.1.1 Consistency and maintenance

A component library within a design system enables the authors to promote a single source of truth more effectively. Consistency in both code and the resulting implementation is key to improving efficiency in development. Providing a consistent UI and UX across distinct, large projects makes it easier to achieve uniformity [45].

Consistent code that follows certain patterns and guidelines is also highly more maintainable, and will result in further time savings in the future. Adding additional features to existing components in the library requires less refactoring if the components have been created following the same conventions.

An important strategy for managing complexity and code organization is **composition**. Component composition is the process of assembling complicated components from smaller, simpler components [20]. A component template can include sub-components, that are managed separately. This separation of code makes it easier to pinpoint and fix issues that may present themselves. Fig. 2.7 illustrates how a `tab group` is composed of smaller, atomic components, `tab` and `tab panel`.

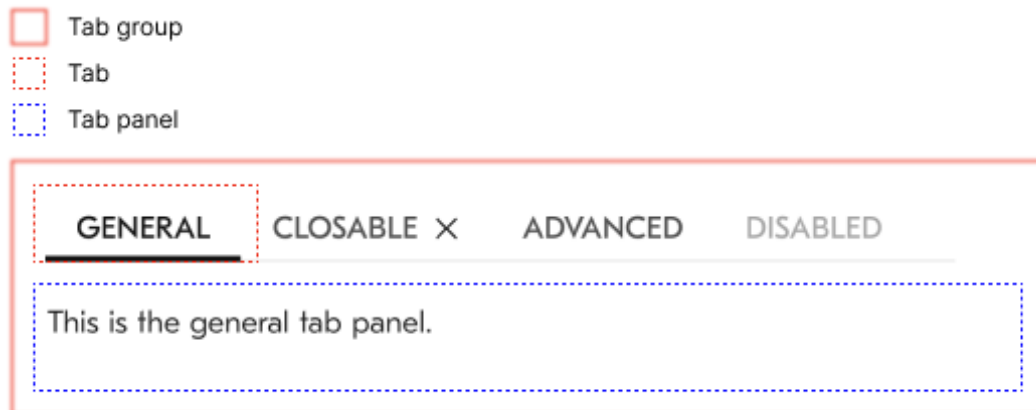


Figure 2.7: Modes UI (see Section 5.1) tab composition.

One way to improve code consistency is by creating guidelines and documenting the workflow of component creation. Another way, that is utilized in almost all larger software projects, is using tools such as linters (e.g., **eslint**) [12]. Linters are tools that take certain

configurations and rules, and then check that all the code abides by those rules. Linting should be configured in the component library itself, to ensure all components follow the same rules. This will create code that is more consistent, readable and maintainable, reduce errors in production, and make the code more secure and performant [46].

Another popular tool for automated code formatting is **Prettier** [33]. Prettier typically runs whenever a file is saved, and saves time and energy in code formatting. Configuring Prettier for the entire component library also ensures there are no clashes in code formatting styles, and arguments over code style will not cause additional overhead in code reviews.

2.1.1.2 Accessibility & compatibility

As web sites and applications have evolved from simple text interfaces to complex visual interfaces, there is a risk that some people will be excluded because they are unable to use traditional methods of access. Even though this change has given people a more creative and flexible experience, there needs to be additional work done to ensure access to those with disabilities [2].

A component library can assist in this aspect as well. The authors of the library should take **accessibility guidelines** (e.g., W3C's [43]) in to careful consideration when developing new components. Once the components are properly built with accessibility as a priority, product teams can use them without having to worry about accessibility, as the component library provides accessibility for all the building blocks of their product.

Cross-browser and cross-device compatibility can be challenging for front-end developers. However, usage of component libraries will go a long way toward preventing incompatibility through standardization. This is especially true for a **technology agnostic** (see [Chapter 3](#)) component library, as using a browser standard enables usage across different tools, frameworks, and even devices.

2.1.1.3 Avoiding code duplication

As complex software systems are developed and maintained, code duplication is a frequent occurrence. During the refactoring phases of a software life cycle, the identification and elimination of duplicate code has evolved into a regular procedure. However, code duplication identification frequently generates a significant amount of data, making it challenging to comprehend the duplication issue as a whole. [36]

Avoiding duplicate code is one of the first things you learn as an aspiring programmer. This should be done proactively, instead of reactively. A design system, more specifically the component library, can prevent unnecessary work from product teams re-creating the same components again and again. Being able to utilize existing components that already have the company branding and consistent functionality saves users a ton of valuable development time.

2.1.1.4 Estimating value in development

For a rough estimation, let us assume that a component library has 20 different components, and the component library is being utilized by 10 different product teams. Even with these fairly conservative figures, this would result in 200 different implementations of the

components if each team were to create their own versions separately. The implementation process of a component is also a long and continuous task that can take anywhere from a couple hours (simple components like buttons), to hundreds of hours (complex components, such as date-time pickers). If we assume an average implementation time of 20 hours, and an average salary of 25€/hr (again, both conservative), the cost of implementing 200 components would be **a hundred thousand euros**. This figure does not even take in to account the continuous maintenance and bug-fixing costs, which will also be multiplied as different versions of the same components have to be maintained.

These figures may not be accurate depending on the adoption of the component library, but they still illustrate the point well. Implementation wise, a component library's value strongly correlates with the amount of users that adopt it. In the hypothetical estimation above, each team could potentially save hundreds of development hours by adopting design system components instead of creating the components themselves. This would also lead to multiple intangible benefits, that will be covered in the next subsection.

2.1.2 Intangible benefits

The previous subsections attempted to quantitatively measure the time-savings and value that a design system can generate. However, some major benefits of a design system can not be quantified, such as consistency in branding and UIs. A design system usually also increases the quality of the work delivered, Bryn Ray estimates the relative increase in quality to be around **18%** in his article estimating a design system's value [35].

2.1.2.1 User experience

UX is a crucial part of any product, and web products are no exception. An intuitive and consistent UI improves the user experience, which results in further value that is indirectly generated by the design system. Components and layouts that are consistent across different products help users switch between Metso Outotec applications and web pages. This can lead to users preferring to use another Metso Outotec product in the future instead of an alternative solution, as they are already familiar it.

Customer retention is extremely important for digital products. While user experience is subjective, the attributes that create the user experience are objective. A good design system follows objectively solid guidelines, which allows users of the design system to create a user interface that provides a good user experience. This in turn makes end-users more likely to prefer Metso Outotec's services in the future, which translates to actual revenue that the design system is responsible for.

Having a common design system eases the workload of designers by providing ready-made components and templates, and as such saves a lot of time for designers. This allows designers to focus their time on improving the UX, providing value for users, and solving their problems [35].

2.1.2.2 Branding

Large corporations spend millions of dollars to improve and maintain their brand image through marketing, and according to a survey by Gartner, marketing expenses equal to

12.1% of a company’s revenue on average [31]. The cost of building and maintaining a design system may be hard to justify initially (more about this later in [Section 2.2.1](#)), but when compared to marketing expenses, the cost of a design system is low. It is difficult to estimate how much a design system contributes in terms of brand recognition, although it definitely is a positive contributor. A design system’s documentation and guidelines are valuable ways to distribute the branding team’s policies and decisions regarding brand identity. Ensuring the correct usage of design tokens such as typography and colors is easier when they are easily available in the design system.

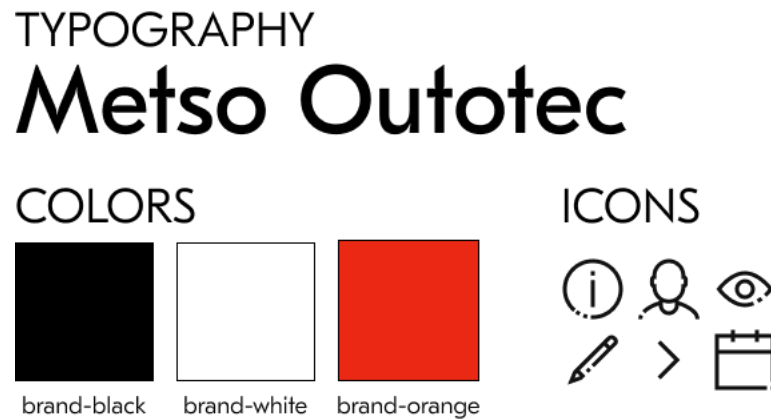


Figure 2.8: Some of the Metso Outotec brand tokens provided by the design system.

These tokens also ease the work of developers, as they can use the provided tokens to ensure consistency in colors, spacing, typography, etc., across different products (see [Fig. 2.8](#)). Clear variables make it easier for designers to document the exact values in their designs, that developers will use to implement the designs. Instead of having to locate and download the on-brand icons and fonts separately, users of the component library can find them bundled in the library, saving time and ensuring correct usage.

2.2 The challenges of design systems

A major challenge in introducing a design system to a large user base is adoption. A design system’s value is largely dependant on its rate of adoption, which means efficiently distributing and sharing information about the design system is imperative. The issue with adoption is two-fold: how to spread knowledge about the design system and its benefits, and how to convince users to adopt it?

The next two subsections will examine issues regarding budgeting, internal communication, and contribution.

2.2.1 Justifying initial investment

As alluded to in the introduction to this section, the numerous benefits of a design system do not come without a cost. The initial creation and maintenance of a design system presents challenges, especially in budgeting and communication. The fiscal cost of creating a design

system is not trivial, as designer and developer hours are expensive and often licensing fees for third-party solutions play a part as well.

Justifying the initial investment to **stakeholders** may be difficult, as the design system does not generate any direct income if used internally. As such, business owners and stakeholders must be convinced about the value proposition through benchmarking and knowledge sharing. The extra value brought on by a design system must somehow be made concrete enough to justify its creation.

Asking stakeholders to reflect on their current product can often highlight the problems that could have been mitigated, or even prevented, by a design system. Is there inconsistency across styling and user experience across different elements of the product? Is onboarding difficult for new hires due to the undocumented complexity of the current solution? Is there a recurring theme of re-creating already existing implementations? Have you had to answer the same question about the system multiple times? All of these questions emphasize the issues that a design system solves. Using relatable examples makes it easier to understand the value of a design system, and *why* investing in it is worth it.

2.2.2 Adoption

As discussed in [Section 2.1](#), the value of a design system directly correlates with the size of the user base that has adopted it. Ensuring mass adoption inside a large company is a difficult challenge. In the start-up world there is a saying that your product needs to be ten times better than what is already out there. The same idea is appropriate for the case of design system adoption, users need to know the value generated by its adoption outbalances the effort to carry out the migration work. There are multiple different factors that complicate the adoption of a new system. This section will go through the adoption issues that are relevant to the introduction of a design system in a large company, and some possible remedies for them.

2.2.2.1 Documentation

Documentation is clearly an integral part of a functioning design system, and its importance in the initial adoption of design system users is also paramount. The documentation must explain clearly to new users the what, why and how of the design system. What it is, why it exists, and how the users can adopt it. Documentation would ideally have a single source of truth that users could refer to, that included both design and development documentation. In reality, the difference in nature between design and development documentation makes such an all-in-one documentation challenging to create. Often design systems have separate documentation for the component library and design files. One is directed towards developers, the other towards designers. In these situations it is important to distinguish between the two, while still keeping them intertwined and accessible to users.

As shown in [Fig. 2.2](#), the Metso Outotec Design System utilizes 3rd party tools Storybook and zeroheight for its documentation. Zeroheight is a tool created to act as a single source of truth, and to accomplish this it has features such as Storybook integration built-in. Their goal is to "connect everyone working with your design system", and with proper usage this can be achieved. The Metso Outotec Design System has benefited from such a tool, as users now have a clear documentation web page they can navigate for any and all

information about the design system. Developers still use the Storybook web page itself for more detailed development documentation, but having a common site makes it much less confusing for new users to navigate the documentation, and to get a clearer picture of the system as a whole.

2.2.2.2 Ease of adoption

The ease of adoption is a surprisingly large factor in improving design system adoption. This aspect is one that the developers of the component library can directly influence through distribution methods of the library package. Ensuring ease of adoption becomes a challenge if and when the component library must be private and only accessible by company internal employees and temporary contractors. Sharing access across numerous different teams and their ever-changing external workers can prove to be tiresome and a manual process. This results in users having to wait for access rights to start developing, and for the design system team to have to use resources to respond to every access right request.

Ideally, the library package would be public, where teams could access it from e.g., the public **Node Package Manager (NPM)** registry. This would ease adoption and save time, while also providing the possibility of improving documentation and demo capabilities with interactive code editor tools. This would also make it easier for users to contribute to the system. However, this can become a legal and security concern inside a larger company, and getting approval for the publication of the component library can range from difficult to impossible. If the package must be private, the component library distribution must be well thought out so that handing out access rights and generating access tokens is well documented and the communication channels to request those rights are clear.

2.2.2.3 Communication and compatibility

The company's internal channels of communication play a big role in spreading knowledge about the design system, as sharing information across a large, global company can be challenging. Different practices and even languages present a bottleneck in the flow of information. A large company is also often split into numerous different teams, value streams, and products, and communication across parties that do not usually collaborate is often minimal or nonexistent. This can result in many, if not most, possible users not even being aware of the existence of a common company design system that could also generate value for them.

Product teams also often already have their own practices and possibly even their own components in place. Therefore, it may be tough for product owners and stakeholders to justify switching to the company's design system, as that requires additional work, while the functionality remains mostly the same. The key idea that must be instilled in the thought process of these teams, is that in the long run a design system will ease the workload of the product team. The components will be maintained by the design system developers, and the product teams' designers can take advantage of existing design documentations and guidelines. Future compatibility will be ensured, and users can contribute their own implementations for the greater good of the whole component library user base.

2.2.3 Maintenance

Maintenance can become an issue in a design system as it grows larger, and technologies evolve. Documentation, designs, and implementation must all be kept in sync, and this can prove to be a grueling manual task. From the perspective of development, the project's dependencies' latest updates may contain breaking changes. Therefore updating the dependencies may cause unforeseen extra work. The team can choose to not update certain dependencies to avoid this issue. However, not updating dependencies can result in security vulnerabilities which cannot exist in a system that is being utilized across the company.

A design system serves a variety of teams, each with different use cases and needs. Creating a system that best serves all the different product teams is challenging, and trying to cover specific needs of a team easily leads to scope creep. This has been an issue with Modes as well, as some DSUI components and features have been specifically created for one product team's needs. This has led to unnecessary complexity in components and as a result, technical debt and bugs. To combat this, components should remain as generic, atomic pieces that product teams can use to build their unique layouts and functionalities (see Fig. 2.9).

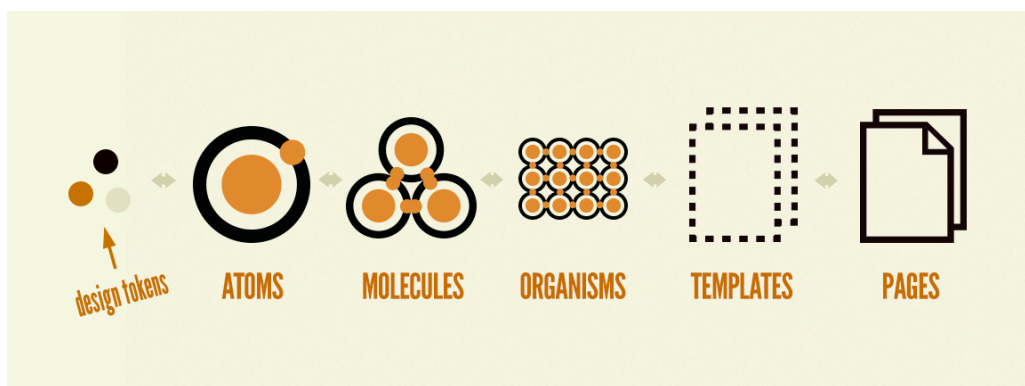


Figure 2.9: Atomic design breaks down the creation of components and layouts into smaller pieces to improve maintainability and re-usability. [15]

2.2.4 Design systems in industrial context

Design systems in an industrial context require extra considerations for the accessibility requirements and features and which technologies are supported. The usage environment and equipment may also wildly vary between users. Some users operate in a dark environment, some with low resolution or small screens, some wear gloves. These factors must be considered when creating the design system to ensure ease of adoption and accessibility for all users.

Industrial context also means that for many components, performance must be prioritized and carefully thought out. Use cases of the component library can often include data streams with vast amounts of incoming data, which needs to be rendered efficiently and without issues. This is especially true for charting components, and as such choosing an existing charting library must be done with that in consideration.

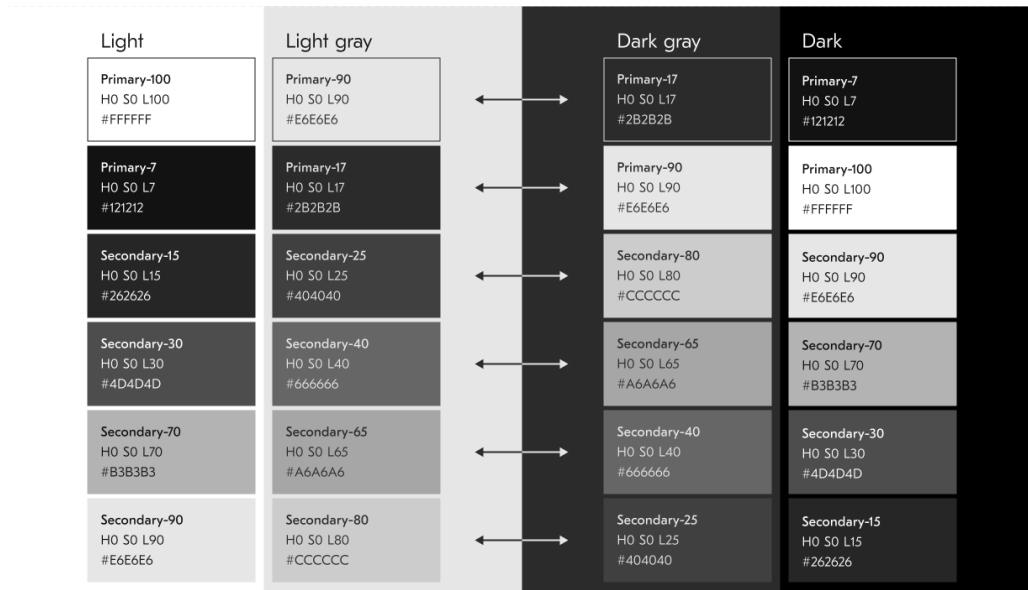


Figure 2.10: Modes offers multiple color themes for different usage scenarios.

The light gray and dark gray color themes in [Fig. 2.10](#) were created at the request of a product team. They needed a dark theme that had lower contrast than the regular dark theme. This is due to the fact that their software is being displayed in a poorly lit environment, from an old and small screen. They also requested for even larger component size options, as users that were wearing gloves and using a touch screen had issues with accuracy on the smaller components. Such complications may not be obvious, but are present in applications where usage conditions prove to be difficult.

As mentioned earlier, digital products in an industrial context may have long lifespans. A team has to consider this when planning for a new product, and one user said the following. "We have been planning the creation of a new digital product. Choosing the correct technologies has been difficult, as the lifespan of our product can exceed 20 years. Ensuring continued maintenance is a key aspect in our project, and we have had concerns over the longevity of React or other proprietary frameworks." This is a common issue among teams creating a new web based product for an industrial application.

TECHNOLOGY AGNOSTICISM

3.1 Introduction to technology agnosticism

Technology agnostic is a relatively ambiguous term, but in computer science it is defined as:

***Agnostic:** Denoting or relating to hardware or software that is compatible with many types of platform or operating system. [30]*

More specifically, in the context of design systems it refers to a component library that is not tied to one specific framework or technology. In other words, the components do not rely on any proprietary technology or framework to function and render properly. In many cases, **framework agnostic** would be a more accurate descriptor in the context of component libraries, as a lot of the technology conflicts are due to the abundance of different front-end frameworks in web development.

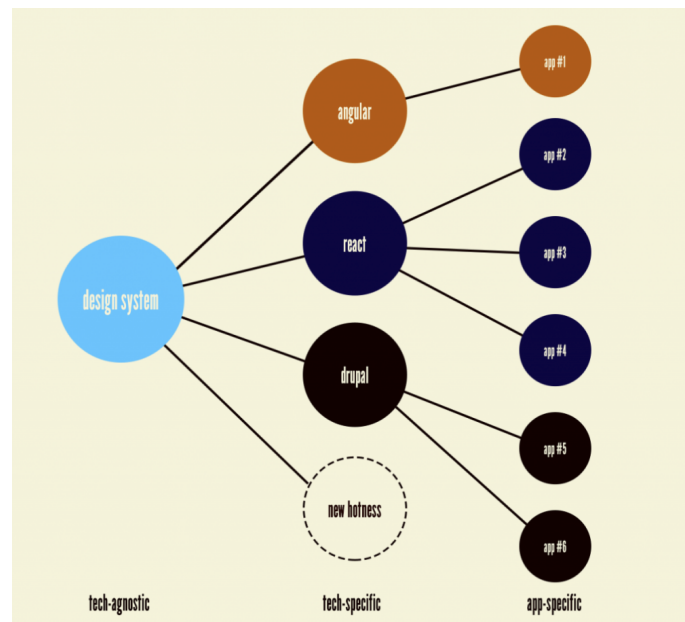


Figure 3.1: An example of a technology agnostic design system.

Fig. 3.1 from Brad Frost’s blog post [21] shows how a technology agnostic design system is able to serve apps built from a number of different technologies. In his post he brings up the idea of creating different variations of the component library for different front-end frameworks. However, that is a lot of work, and since his post in 2017, there has been a lot of progress in web standards and new technologies that allow for the creation of a single component library that works with any framework.

Most component libraries available online are tied to a popular front-end framework, such as MUI (formerly Material UI), that is only accessible for React projects. However, more and more technology/framework agnostic component and UI libraries have begun emerging as the industry has started to realize the long term benefits of technology agnosticism. In fact, **Google** (the creator of MUI), also now offers an alternative component library that is based on Web Components that is gaining popularity and is being developed actively. **Adobe's** Spectrum Web Components and **Salesforce's** Lightning Web Components are both also good examples and indicators about the future of the industry. **Microsoft** is also investing in to a technology agnostic component library, with Fluent UI and FAST.

In practice, with a technology agnostic component library, a team starting a new project can choose the technology that fits their project best, without having to worry about compatibility with the design system. Some product teams and applications may prefer using e.g., **Angular** or **Vue.js** over **React**, as they either have more expertise using those, or the properties of those technologies better suit the product they are creating.

In addition to causing problems for subsequent developments, these frameworks can bloat the library unnecessarily. They may have features and functionalities that are not needed for the project. Unfortunately, users must still download the framework's unused JavaScript and CSS, which makes pages load more slowly and hinders the user experience. On the flip side of that coin, frameworks might not go far enough, necessitating a significant amount of custom code from developers to accomplish the objectives of their projects. At some point, a threshold is exceeded where the time spent altering, extending, and fixing the framework outweighs the initial benefits of utilizing it. [15]

Currently in Metso Outotec the component library (**DSUI**) is React only, which limits the options of product teams if they want to take full advantage of **Modes**. According to an internal survey, 19% of product teams cannot fully utilize Modes due to this limitation, and some reported that they might have chosen a different technology if this limitation did not exist. As Metso Outotec is a large corporation, new digital products and web pages are created regularly. Having a technology agnostic option should improve the adoption rate of the design system and strengthen branding.

React is a mature framework with an enormous user base, and it has served most product teams well. However, as it is still a proprietary technology from a third party, its future support and development is not guaranteed. Future updates or versions can also lead to breaking changes that can require substantial effort to correct in the affected components. This also means that the component library's lifespan is limited to that of the framework's. There is a continuous wave of new front-end frameworks emerging, and React might not be the king of the hill for long. New frameworks such as **Svelte** and **Solid** already had higher user satisfaction last year, and are quickly growing in popularity [41].

To combat these issues, the component library of a design system should be created using a standard that will be supported for the foreseeable future. In this case, that standard is **Web Components** and the technologies it encapsulates. The following sections will inspect the theoretical aspect of Web Components, and the challenges and benefits its implementation provides.

3.2 Web Components

Web Components are a collection of technologies that enable the creation of reusable custom elements that are functionally isolated from the rest of the HTML structure, and that can be utilized inside web applications. Web Components consist of a few main technologies (see [Fig. 3.3](#)), which can be used together to create these versatile and standardized components that can be reused across an application without fear of code collisions [26]. They are a part of the **browser standard**, which ensures that they will be supported in the long term. One of the main appeals of Web Components is their inherent cross-framework capabilities, which makes them apt for component libraries that are used by multiple different projects and teams.

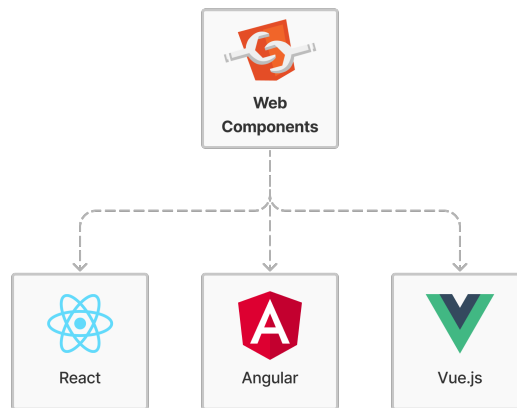


Figure 3.2: Web Components can be used across different frameworks.

Despite Web Components being under development by Google since at least 2011 when they were first introduced to the public, they have not gained as much traction as many would have hoped. This is due to a variety of different factors, from the rise of Facebook’s React in the same time-frame, to the technological shortcomings that Web Components inherently had. It took until 2016 for the Web Component standard to be widely adopted in modern browsers, while popular modern frameworks such as React and Angular offered more features earlier. At the time, React and other similar frameworks had already created a tectonic shift in web development, and there was now less of a need for Web Components.

The technological issues with Web Components are mainly related to handling of events and the flow of data, where established frameworks have often been built around optimizing these factors. The Shadow DOM (see [Section 3.2.2](#)) does not function out-of-the-box with native forms, and *autonomous* Web Components (see [Section 3.2.1](#)) cannot extend `HTMLInputElement`, due to the Safari team deciding not to support customized built-ins. This means using Web Components inside forms requires workarounds, such as appending the values of the input fields to the `formData` object on form submission (see [Section 5.1.3.2](#)).

An issue that is especially relevant to the context of component libraries, is that creating

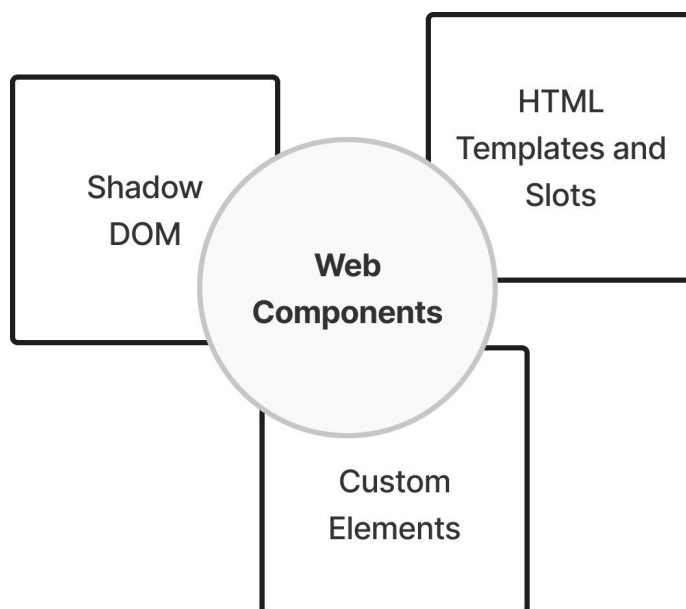


Figure 3.3: The technologies behind Web Components.

many Web Components using just the native standard is quite tedious and results in lots of boilerplate code. Web Components were created to eliminate the need for frameworks, but to build them efficiently, ironically, authors still need to use a framework. Fortunately, Web Component libraries, such as Google's **Lit** and Ionic's **Stencil**, fix a lot of these issues and improve the component creation and maintenance process of developing with Web Components significantly. These libraries will be discussed further in [Section 3.4](#).

3.2.1 Custom Elements

Custom Elements offer authors a way to create their own fully functional DOM elements. Authors have always had the option to include non-standard elements in their documents with application-specific behavior added later via scripting or a similar method, but historically these elements have been non-conforming and not very useful. By defining custom elements, authors can instruct the parser on how to generate an element and how members of that class should respond to changes. [8]

Custom Elements can be divided into **autonomous custom elements**, and **customized built-in elements**. The difference in code is minimal, but the implications are significant. An autonomous custom element *always* extends `HTMLElement`, while customized built-in elements can extend, e.g., `HTMLButtonElement` (see [Code example 3.1](#) and [Code example 3.2](#) for a code comparison). This means that autonomous custom elements cannot directly inherit some important functionality, and the browser in some cases will treat them differently to built-in elements, e.g., in native forms as discussed in the previous [Section 3.2](#).

In an effort to "rationalise the platform", custom elements are used to describe lower-level author-exposed extensibility points (like the definition of a custom element) in terms of higher-level platform characteristics (like the components of HTML). Although custom elements currently have several functional and semantic limitations that prohibit them from

TypeScript

```

class DataTable extends HTMLElement {
  constructor() {
    // Always call super first in constructor
    super();

    ...
  }
  // element functionality and life-cycle methods here
}

// use created class to define the element for the browser
customElements.define("data-table", DataTable);

```

HTML

```

<!DOCTYPE html>
<data-table></data-table>

```

Code example 3.1: An autonomous custom element.

TypeScript

```

class FancyButton extends HTMLButtonElement {
  constructor() {
    super();
    this.addEventListener("click", () => {
      // define customized behaviour
    });
  }
  // element functionality and life-cycle methods here
}

// use created class to define the element for the browser
customElements.define(
  "fancy-button", FancyButton, { extends: "button" }
);

```

HTML

```

<!DOCTYPE html>
<button is="fancy-button"></button>

```

Code example 3.2: A customized built-in element.

fully describing the behaviors of HTML's existing elements, the gap is planned to be closed over time [8].

3.2.2 Shadow DOM

Shadow DOM is an extension of the regular **DOM** (Document Object Model). The DOM is a programming interface for web documents, which allows software to alter the document's structure, appearance, and content, by serving as a representation of the page. Programming languages can communicate with a page by interacting with the nodes and objects that the DOM uses to represent the document. It is a representation of the web page that is object-oriented and can be changed using a scripting language such as JavaScript. [23]

The ability to keep the HTML structure, style, and behavior hidden and distinct from other code on the page so that various parts do not clash, is called **encapsulation**. It is a crucial component of web components. This allows for easier code maintenance and less conflicts [25]. A crucial component of this is the Shadow DOM API, which offers a method of connecting a hidden separated DOM to an element (see Fig. 3.4).

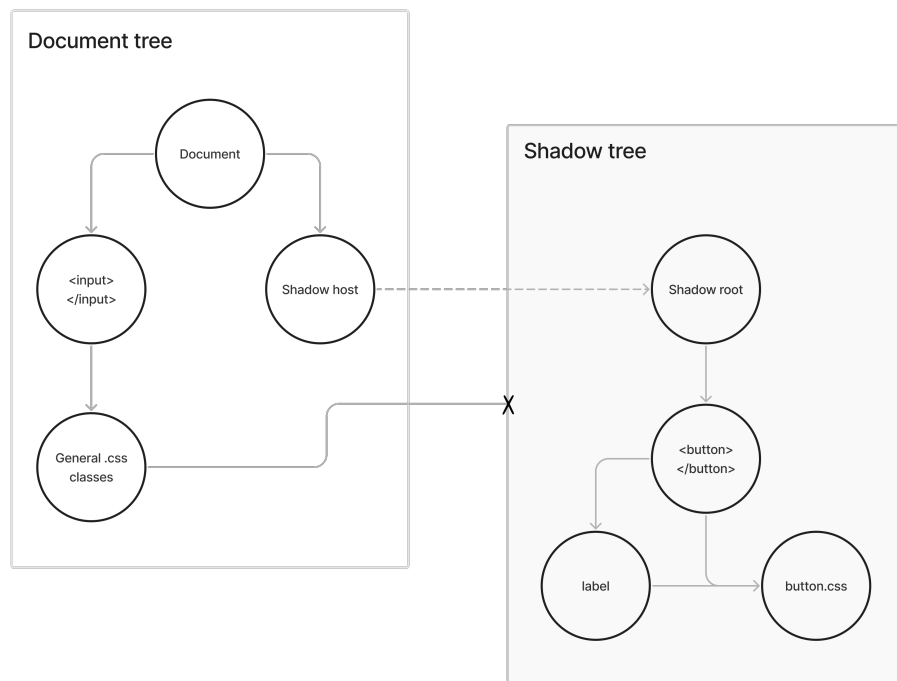


Figure 3.4: The shadow tree encapsulates styling and functionality.

Shadow host	The regular DOM node that the shadow DOM is attached to.
Shadow tree	The DOM tree inside the shadow DOM.
Shadow root	The root node of the shadow tree.

Table 3.1: Legend for Fig. 3.4.

3.2.3 HTML Templates & Slots

HTML templates use **template literals** (delimited with backticks ```) to allow multi-line strings, embedded expressions through string-interpolation, and tagged templates [24]. The `<template>` and `<slot>` elements in Web Components enable authors to write markup templates that are not displayed in the rendered page. These templates can then be reused multiple times as the basis of a custom element's structure.

Components can have named slots, and a default slot. Any elements that are direct children of a Web Component will be slotted in the default slot, unless they are specifically defined to go in to a certain slot (e.g., `<mo-icon slot="prefix" name="settings"></mo-icon>`). In the example below (Fig. 3.5), the label is the default slot, and content can be slotted before or after it using the `prefix` and `suffix` slots. In the button these elements are typically icons, but users are able to add any elements in the slots. The component might not look correct with e.g., some large content though, and thus some of the responsibility to create functional configurations of the `<mo-button></mo-button>` falls to the user.



```
html`
  ${
    this.hasSlotController.test('prefix')
    ? html`<span part="prefix" class="button__prefix">
      <slot name="prefix"></slot>
    </span>`
    : ''
  }

  <span part="label" class="button__label">
    <slot></slot>
  </span>

  ${
    this.hasSlotController.test('suffix')
    ? html`<span part="suffix" class="suffix">
      <slot name="suffix"></slot>
    </span>`
    : ''
  }
`
```

Code example 3.3: Slotting content in HTML templates.

The `<slot></slot>` in the code example above specifies where the content in the default slot will be slotted. The `prefix` and `suffix` slots check if the user has input any content inside them, and render only if they have. Testing for slot content in this fashion is not default Web Component functionality, and in this example it is a separate helper class. This ensures empty slots do not take up space in the element. This is done using the conditional (ternary) operator, which executes an expression based on if the first operand is *truthy* or *falsy*. If *truthy*, it executes anything after the `?` character, if *falsy*, anything after the `:`.

An empty string (‘’) will not render anything.

```
<mo-button variant="secondary">
  <mo-icon slot="prefix" name="settings"></mo-icon>
  <p>Settings</p>
</mo-button>
```

HTML

Code example 3.4: Slotted button example from Modes UI (see [Section 5.1](#)).

In the above example `<p>Settings</p>` is slotted into the default slot, giving the button a label. An `<mo-icon></mo-icon>` is slotted into the `prefix` slot, see the first example in [Fig. 3.5](#) for the rendered component. Such components follow *atomic design* (see [Fig. 2.9](#)).



Figure 3.5: A button with different slots.

Templates and slots facilitate the creation of flexible and customizable, *composable* components. **Composability** is a principle in system design that concerns component relationships with one another. A component library inside a design system should ideally be highly composable, i.e., the components can be chosen and assembled in numerous combinations to satisfy different user requirements. Lego pieces are a common analogy to describe such a system, as all Lego pieces can connect to one another. As a result, they allow for tremendous creative freedom through numerous permutations in configuration.

3.3 Support, migration & adoption

Technology agnosticism and Web Components evidently present a great deal of added value to a design system, but as they are a relatively new standard, their adoption may not be trivial. Browser support, specific technology compatibility and migration documentation are topics that must be examined and covered thoroughly before attempting to create a technology agnostic component library. This section will discuss these topics, with a focus on the case study of Metso Outotec and how these issues may arise in this specific context.

3.3.1 Support

Web Components are supported by all modern browsers, but that does not necessarily mean they work for every user. Internet Explorer (IE) has been the bane of web developers for a long time now as supporting it is often challenging, but according to recent data, less than 1% of users are actually using IE in 2021 [3]. According to *caniuse.com*, 77.73% of

users are using modern browsers with full Custom Elements support, plus 18.36% of users have full support with some small deficiencies [4]. This totals to 96.09% of the global user base, which is sufficient to adopt this feature in a large company such as Metso Outotec. Furthermore, Metso Outotec takes security seriously, and one aspect of that is ensuring employees update their browser regularly to fix security issues discovered in past versions.

At the time of writing, Safari and older Chrome versions have partial support (see Table 3.2): they support "Autonomous custom elements" (e.g., `<mo-button></mo-button>`), but not "Customized built-in elements" (e.g., `<button></button>`, that has been customized). However, in the case of a component library built using Web Components, this should not cause any issues as the components are autonomous custom elements.

Browser	Versions
Google Chrome	Partial support: v. 54-66 (Oct 12, 2016 - Apr 18, 2018) Full support: v. 67 -> (May 29, 2018 -)
Microsoft Edge	Full support: v. 79 -> (Jan 15, 2020 -)
Mozilla Firefox	Full support: v. 63 -> (Oct 23, 2018)
MacOS Safari	Partial support: v. 10.1 -> (Mar 27, 2017 -)

Table 3.2: Web Component browser support table [4].

As discussed in earlier chapters, most product teams are already using a combination of different technologies, with components and a workflow often built specifically for those technologies. Inside Metso Outotec (and in general), React is the most common front-end framework among product teams. In addition to browser support, it is not a guarantee that a specific technology (such as React) fully supports web components out of the box. Indeed, the current version of React used by most teams does not have full web component support yet, but it is available as an experimental feature that should be released in the next major React update. This would cause additional headaches if product teams that are currently using React were to migrate to a technology agnostic version of the component library that did not consider this issue. For those teams, migrating to a web component-based component library would be a sizable, manual task that could be hard to justify.

3.3.2 Migration & adoption

In order to establish large-scale adoption, the technology agnostic component library must ensure ease of migration. To achieve this, the new component library should prioritize compatibility with React, and it should include exhaustive documentation on the migration process. Although the current version of React does not have full web component support, there are approaches that can be taken to provide a version of the component library that is compatible with React out of the box. One such approach is utilizing a utility wrapper to automatically generate a fully-fledged React component in the build phase of the library (see source code example in Fig. 5.3). Google's Lit (a simple library that improves the creation process of web components, see section 3.4) provides a package (<https://www.npmjs.com/package/@lit-labs/react>) that offers this functionality, and employing

it should theoretically enable React users to easily adopt the technology agnostic component library despite the lack of full React support for web components.

Migration guides in software engineering are a generic term that usually refers to documentation that will guide users on migrating from to a newer version of the same package or library. These guides usually contain breaking changes in functionality and syntax, but usually making these changes does not require major refactoring. Migration from one library to another is a considerably heavier task, and as such requires more documentation. Migration should be systematic and consistent for all product teams and the libraries' components, so that it can have detailed and accurate documentation that applies on a general level.

3.4 Choosing a Web Component library

Web Components can be difficult to write from scratch, and writing a component can require a lot of boilerplate code. This becomes an issue especially when creating numerous different custom Web Components, as is the case with a component library. Fortunately, libraries have been created that help with this creation process, making the workflow more straightforward and efficient, and saving developers time and effort [44]. In this section, some of these libraries are compared to and assessed on their value in the context of a component library for a large design system.

Despite web components being a fairly new standard, there are multiple existing libraries to choose from to have a solid foundation to start creating the new component library on. These libraries offer a level of abstraction above plain web components that makes it easier to develop and maintain for larger projects and more complex components. They could be described as transpilers (also known as source-to-source compilers) that take code in their own syntax, in JavaScript or TypeScript, to generate web components (custom elements specifically) for the browser to use.

As with the selection of any technology or tool in web development, it is important to ensure the web component library has a sufficient number of functionality to address all possible use cases of the project. Is the library interoperable, i.e., does it reveal implementation details? Web components that are created by a good web component library should "simply work" like any other HTML element. Good libraries also offer a lot of value relative to their download size, i.e, they have a high value-to-payload ratio. In [Table 3.3](#) there is a collection of libraries that support ES modules, Custom Elements, the Shadow DOM, and HTML Templates. [44]

Library	Description
Hybrids	a UI library for creating web components with simple and functional API. The library uses plain objects and pure functions for defining custom elements, which allow very flexible composition. It provides built-in cache mechanism, template engine based on tagged template literals, and integration with developer tools.
Lit	a web component library built by Google, with a simple element creation API. Lit offers one- and two-way data binding into element templates, and provides shims for better cross-browser performance.
Skate.js	a library built on top of the W3C web component specs that enables you to write functional and performant web components with a very small footprint. Skate is inherently cross-framework compatible.
Slim.js	a lightweight web component library that provides extended capabilities for components, such as data binding, using es6 native class inheritance.
Stencil	is an opensource compiler that generates standards-compliant web components.

Table 3.3: A comparison between the most popular Web Component libraries.

This list has been compiled by the community and can be modified via pull request under the community repository for this website [44]. Projects are listed alphabetically and are known to support all four aspects of the web components specification. **Lit** and **Stencil** are highlighted as they are the top options and they will be examined in the next subsections.

3.4.1 Web Component compilers

Stencil is slightly different from the other options in [Table 3.3](#), as it is more of a compiler, rather than a library. Stencil is built and maintained by Ionic, and is powering design systems and cross-framework components in major companies such as Apple, Amazon and Microsoft. Stencil claims to be built in a way that is optimized towards design system usage specifically, and they even offer a Stencil Enterprise package for companies that wish to have support on creating a design system using Stencil.

A similar tool to Stencil that enables the creation of a framework agnostic component library is **Svelte**. Svelte is a JavaScript framework, that was designed to create reactive applications with minimal overhead. Both tools allow authors to create with a framework and compile to standalone Web Components [13]. These kinds of tools may well be the future of frontend development.

3.4.2 Lit

Lit is created and maintained by Google, and is the successor of the pioneer library in Web Components, Polymer. It is a simple library for building fast, lightweight Web Components. At its core it is a base-class aimed at removing boilerplate that provides scoped styles, reactive state and a declarative template system [20]. Lit components are always standard Web Components, that have native browser support and have the power of interoperability.

These features make Lit an excellent choice for the creation of shareable components, that can be shared through a design system. Lit improves the Web Component creation process significantly without introducing significant overhead. Lit has first-class TypeScript support, which can be utilized to make components created using Lit easier to maintain and update.

When developing, Lit doesn't require compilation or building, so if you'd rather, you can use it virtually tool-free. Tools for production (translation, template minification, etc.) and first-rate IDE assistance (code-completion, linting, etc.) are easily accessible. [20]

Lit was chosen as the library to improve Web Component development for the new technology agnostic component library inside Metso Outotec. The aforementioned features suit the needs of Metso Outotec, and as it is maintained by Google, it has exhaustive documentation and utilities. [Section 4.2](#) will dive deeper in to the development aspects of using Lit, and [Section 5.1](#) will present the results of this implementation.

IMPLEMENTATION

4.1 Component library management

As mentioned before, the implementation side of a design system is typically a component library. This component library consists of multiple stand-alone components, that can usually be used and imported individually ("cherry-picked"), to maintain minimal bundle size in the actual web application. A component library can also include helpful utility functions and design system specific logos and fonts. As such, a component library can quickly become large in complexity and size, and managing assets, components, and different packages becomes an important problem to solve.

The components in a component library generally have dependencies to each other and external libraries and packages. Managing these dependencies and ensuring the versions are correct require a **package manager**. The development of the present massive number of server-side Node.js and client-side JavaScript apps is facilitated by a big repository of JavaScript-based software packages, which is accessed through the **node package manager** (npm) [48]. Since its beginning six years ago, npm has expanded to become one of the biggest software ecosystems, hosting more than 230 000 packages and seeing hundreds of millions of installations each week [48]. **Yarn** is a popular alternative to npm, that promises better performance and more instantaneous and stable installations. However, since npm 5 was released, there is little difference between the package managers, and it comes down to preference and what is already being used. npm is currently still more popular, but the gap is closing [28].

In addition to package management, a component library needs version control. The industry standard version control system is **Git**. Git provides the necessary framework to handle changes in code across multiple contributors, while also having control over what is released to users and when. However, a component library also needs a layer of abstraction on top of Git, such as **GitHub** or **Azure DevOps**, for access control, distribution (see [Section 4.1.3](#)), and peer reviews.

Another important feature for a component library is **bundling**. JavaScript bundling is a method of optimization, that allows for a reduction in requests for JavaScript files by merging multiple files into one file. This reduces the number of page requests, resulting in a faster loading time for a web page. As a downside, the browser downloads the bundles synchronously, which means that page rendering is blocked until all bundles are downloaded. However, the time saved by cutting down on server requests by a significant factor generally offsets the cost generated by synchronous bundle downloading. Bundling reduces the overhead created by the component library, and improves the user experience as pages load faster. **Webpack** is by far the most popular JavaScript bundler as it is the oldest, but other bundlers such as **esbuild** have grown in popularity [28], as they offer additional features, and significantly improved performance [11].

The next subsections will cover testing and documentation of a component library, and the options that authors have.

4.1.1 Testing

In order to ensure a good user experience for both end-users and users adopting the component library, every component should have proper testing in place. Testing should be thorough, and running the tests should be part of the release process. Due to the relationships between different components, some changes can have unforeseen repercussions that can be hard to spot manually. Automated testing will catch these issues before the new changes are pushed to a release, and the version that is available to users remains functional.

4.1.1.1 Unit testing & end-to-end testing

Unit testing and end-to-end testing are the two major ways to ensure component functionality stays consistent and correct when the code is modified. End-to-end testing is a newer practice, that attempts to simulate how an actual user would use the components. Unit testing is typically used for testing smaller parts of components individually, and is usually faster to execute. Unit tests are also often faster and simpler to create, and they do not require any external dependencies. End-to-end testing is more suited to testing a product or project with a front- and back-end, and as such unit tests are more common in component libraries.

TypeScript

```
import { expect, fixture, html } from '@open-wc/testing';
import type MOButton from './button';

describe('when provided no parameters', () => {
  it('should render as a <button>', async () => {
    const el = await fixture<MOButton>(
      html` <mo-button>Button Label</mo-button> `
    );
    expect(el.shadowRoot!.querySelector('button')).to.exist;
    expect(el.shadowRoot!.querySelector('a')).not.to.exist;
  });
});
```

Code example 4.1: A simple unit test.

For JavaScript, Jest is the most popular unit testing library, while Cypress and Playwright are commonly used end-to-end testing libraries. For Web Components, Open Web Components offers a testing library that is best suited for Web Components specifically [29]. Its syntax is similar to that of Jest. With unit tests, there are two options on folder and file structure, either the tests are in a separate **tests** folder, or each component folder includes the tests, styles, and the component itself. In component libraries, it makes sense

to group the tests along with the other files related to that component, especially if there are not multiple different test files.

4.1.1.2 Testing best practices

Whether the project uses unit tests, end-to-end tests, or both, it is important to conduct testing in a systematic way. Tests should be kept easy to read, modify, and maintain. Complex setups and many layers of abstraction should be avoided when creating test scenarios. Two systematic ways to improve the quality of unit tests are the three-layer system, and the AAA (Arrange, Act, Assert) pattern. The three-layer system ensures that test descriptions are simple and easy to understand. [18]

- Layer 1: Unit or test requirement that will be tested
- Layer 2: Specific scenario or action to be tested
- Layer 3: Description of the expected result

The AAA pattern refers to the structure within the actual test code. All the setup code and test data required to recreate a test scenario are included in the "arrange" section. Second, the "act" portion does what its name suggests: it runs the unit test. Typically, only one or two lines of code are needed to execute a test. Finally, the "assert" section collects all assertions in which the received output is compared to the expected output. [18]

In the case of end-to-end testing it is important to consider what are the most common and important workflows of the product that is being tested. For a company such as AirBnB, their most important workflow is likely finding and making a reservation on a space or property [10]. Therefore the end-to-end tests should cover this scenario well and ensure that there are no issues. Ideally, tests are broken down in to smaller atomic pieces, to ensure maintainability and easier troubleshooting. These tests should also be realistic, i.e., load on the server or other possible hindrances that occur in real-world use should be taken in to account. Coverage is another important factor, end-to-end tests should cover as many different usage scenarios as possible.

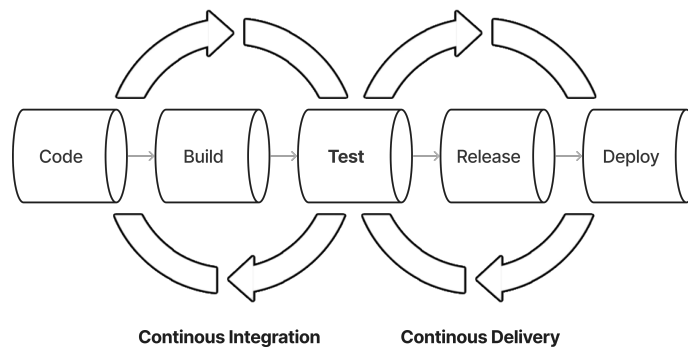


Figure 4.1: A CI/CD Pipeline.

These tests should be run each time a new change is made to ensure previous functionality is still intact. To make sure the tests are run when necessary, they need to be run automatically. **Continuous Integration and Continuous Delivery (CI/CD)** have become standard practice in DevOps to deliver new features fast, and with proper testing [34]. CI refers to splitting a project into smaller coding tasks, and merging the smaller parts on a continuous basis. These builds are combined in a centralized branch or repository, where further testing and automation can take place. CD takes these builds and extends the process with automated releases. If these builds pass tests and are approved, they will automatically be released to production, where they are available for deployment in the real-world. See [Fig. 4.1](#) for an example of a CI/CD pipeline where testing is integral.

4.1.2 Dependencies

Choosing the correct dependencies to use and reasoning their inclusion well is important for any software project. Introducing new dependencies increases the bundle size and can create additional issues with conflicts from different libraries. Especially for a component library that will have many different users with varying needs and requirements, the dependencies must be thought out extra carefully. Another important point to consider is that dependencies themselves often have their own list of dependencies, which means the total amount of packages included can rapidly grow. Minimizing bundle size and maximizing performance should always be the goal when adding new features.

These dependencies can be separated into two groups [27]:

- **"dependencies"**: Packages required by the application in production.
- **"devDependencies"**: Packages that are only needed for local development and testing.

As **"dependencies"** will have to be downloaded and installed by any user that installs and uses the component library, there needs to be adequate evidence that their inclusion is worth it and necessary. Some users may only need a small subset of components, and redundant dependencies can even prove to be a bottleneck in use cases where memory is limited or performance is of utmost importance.

Packages related to improving the development, documentation and testing workflow are in the **"devDependencies"** category. These dependencies **will not** be included in the production builds of users, which means that it is less critical to optimize bundle size and performance. However, inclusion of additional development dependencies should be done for a good reason, as a larger list of dependencies is harder to maintain and keep up to date.

4.1.2.1 Updating dependencies

Dependencies need to be updated periodically to ensure they include the latest stability and security updates. Outdated dependencies can cause issues when adding new dependencies, or updating existing ones. Latest updates often also include useful new features and performance updates, that improve the project overall. Bug fixes are released often in popular libraries, and using an old version could lead to problematic debugging as a bug

that is part of a dependency will be harder to pinpoint. Therefore keeping dependencies up to date should be done systematically and with enough frequency.

However, introducing new package versions in systems with a lot of dependencies may rapidly turn into a headache. Version lock (the inability to upgrade a package without having to release new versions of every dependent package) will be a risk if the dependency specifications are overly restrictive. If dependencies are defined too loosely, version promiscuity (assuming compatibility with more future versions than is reasonable) will ultimately cause issues. Version lock and/or version promiscuity will eventually prohibit authors from simply and securely pushing their project forward. Semantic Versioning (SemVer) provides a simple set of rules that dictates how version updates should happen. [37]

Updating a dependency to a major version may lead to breaking changes, and major version updates often should be done individually. Updating to minor versions or patches is simpler, as they should not introduce breaking changes. In this case it makes sense to have a workflow that automates the updating to minor versions, while requiring oversight for major version updates. SemVer offers ways to limit updates in dependencies to a certain subset of currently available updates.

Manually going through each dependency is usually not feasible. Fortunately, npm offers useful built-in commands such as `npm outdated` and `npm update`. `npm outdated` provides a summary of dependencies that have an update available, showing the current installed version, and the latest minor and major versions. Running `npm update` afterwards will automatically go through all packages that have updates available and install them. However, `npm update` will not update to a major version that comes with breaking changes if the packages are tagged with `.`. The latest versions can be installed manually by using `npm install package@latest`.

For better customization and a more advanced upgrading experience, `npm-check-updates` is an external package that combines the features of the previous two commands and adds further functionality on top of them.

4.1.3 Documentation & distribution

It is imperative to carefully consider documentation and distribution when creating a component library to be used in a design system. There is a wide variety of options of tools that can make it easier to document and distribute the component library. The most commonly used documentation tool for component libraries is **Storybook**, which is what DSUI uses. Storybook allows interaction with the state of the component, and offers addons and plugins that can further improve the documentation. Each component has its own *stories*, that are manually created by the authors to demonstrate different variations of the component.

Another option is to create a static documentation website, which allows for more customization and styling and the open-source ecosystem provides various helpful tools for this scenario as well. Creating a custom documentation website often requires more effort, and does not offer interactive component state configuration for users in the way e.g., Storybook does. The lack of interactivity can be remedied by integrating support for a component playground / interactive code editor, such as CodePen or CodeSandbox. These

are separate websites that offer an IDE-like experience in the web browser, and allow users to try out the components without downloading the package and setting up a project.

Modes UI's (see [Section 5.1](#)) documentation page is created in this way, using a tool called **docsify** [9]. Offering common variations in the docs can be more beneficial for users than segmenting demos and code examples into separate tools such as Storybook. This encourages exhaustive documentation, streamlines development for maintainers, and simplifies the project's structure. It also has a significant improvement on installation and startup times for contributors. As Web Components render natively in the browser, there are no additional frameworks needed to showcase the components.

Distribution of the component library package is generally done through the NPM registry. By default packages added to the registry are public, and do not require additional authentication to download. However, for a design system that is used within a single company, there is often need for authentication through private packages. To share packages with a limited set of users or teams, authors can publish private user-scoped or organization-scoped packages to the npm registry [27]. Package distribution configuration is often handled using a tool that is related to the version control system that the company employs (GitHub, Azure DevOps, etc.).

4.2 Creating components with Lit

As discussed in [Section 3.4](#), Google's Lit is a simple library that builds on top of web component standards to improve the creation and maintenance process of Web Components, with syntax that is similar to other modern component-based web development frameworks. This section will go through the Lit component creation process. This introduction includes a small code example of a simple Hello World component created using Lit. Much of the content of this section is from Lit's extensive documentation at <https://lit.dev/> [20].

Lit (TypeScript)

```
import {html, css, LitElement} from 'lit';
import {customElement, property} from 'lit/decorators.js';

@customElement('simple-greeting')
export class SimpleGreeting extends LitElement {
  static styles = css`p { color: blue }`;

  @property()
  name = 'Default name';

  render() {
    return html`<p>Hello, ${this.name}!</p>`;
  }
}
```

Code example 4.2: Hello World component in Lit.

[Code example 4.2](#) shows how simple component creation can be with Lit. It includes highlighted sections for component definition, styling, reactive properties, and rendering. The following subsections will examine these different properties.

HTML

```
<head>
  <script type="module" src="./simple-greeting.ts"></script>
</head>
<body>
  <simple-greeting name="World"></simple-greeting>
</body>
```

Code example 4.3: HTML to import and render the component.

[Code example 4.3](#) shows an example on how the Lit component can actually be imported and rendered in an HTML file. It is possible to include import and render these same Lit components in a React, Angular, or Vue project, or to add additional tools such as SCSS to ease development. However, this example covers just the basic Lit component creation process, using TypeScript and CSS and rendering them in plain HTML.

4.2.1 Defining a component

Decorators (prefixed with `@`) are used in Lit to offer declarative APIs for various things such as element registration, reactive properties, and queries. Decorators are an experimental TypeScript feature that "provide a way to add both annotations and a meta-programming syntax for class declarations and members" [42].

Lit components can be defined by creating a class that extends `LitElement` and registering that class with the browser:

Lit (TypeScript)

```
@customElement('simple-greeting')
```

Here, the `@customElement` decorator is equivalent to calling `customElements.define` (as in regular Web Components, see [Code example 3.1](#)), which can be used to register a custom element class with the browser and then associating it with an element name (`simple-greeting` in this example). A Lit component is a custom HTML element, so it can be used in the same way as any built-in HTML element, as seen in [Code example 4.3](#).

The `LitElement` class is a subclass of `HTMLElement`, and thus it inherits methods and properties that a standard `HTMLElement` would have. More specifically, a `LitElement` inherits the `ReactiveElement`, which implements reactive properties (see [Section 4.2.3](#)) on top of the `HTMLElement` base class (see [Fig. 4.2](#)). [20]

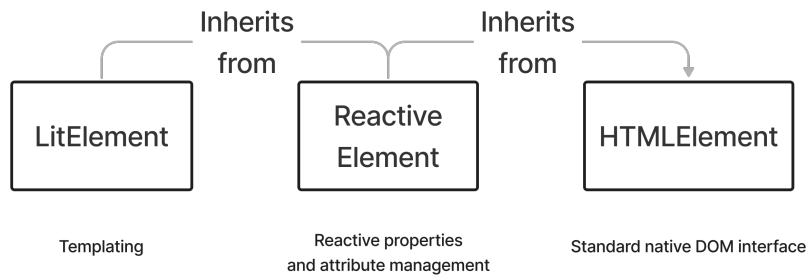


Figure 4.2: LitElement class inheritance.

4.2.1.1 Providing TypeScript typing

The example component in [Code example 4.2](#) can be extended to add additional TypeScript typing information, which improves automatic type checking. The authors of Lit advise publishing the project's `.d.ts` typings in its npm package and adding an `HTMLElementTagNameMap` entry for each element written in TypeScript (see [Code example 4.4](#)) [20].

```
declare global {
  interface HTMLElementTagNameMap {
    "simple-greeting": SimpleGreeting;
  }
}
```

Lit (TypeScript)

Code example 4.4: Additional TypeScript typing for a Lit component.

4.2.2 Scoped styles

The template for each Lit component is rendered to its shadow root. Each component's styles are automatically scoped to the shadow root and only apply to elements in the component's shadow root. As discussed in [Section 3.2.2](#), Shadow DOM offers reliable style encapsulation. Shadow DOM allows Lit to guarantee that any selector you create will only apply to elements in the shadow root of your Lit component.

In Lit, scoped styles are defined in the static `styles` class field using the tagged template literal `css` function [20]:

Lit (TypeScript)

```
static styles = css`p { color: blue }`;
```

Alternatively, styles can be separated in to a distinct module, that exports `css` styles as a tagged template literal:

Lit (TypeScript)

```
// simple-greeting.styles.ts

import { css } from 'lit';

export const greetingStyles = css`
  .redGreeting {
    color: red;
  }
  .bigGreeting {
    font-size: 48px;
  }
`
```

Code example 4.5: An external module that exports a tagged template literal.

Separating styles from the component logic keeps the code clean and easier to read and maintain. These styles can then be imported like any other module and assigned to the `static styles` field, see [Code example 4.6](#).

Lit (TypeScript)

```
// simple-greeting.ts

import { greetingStyles } from './greeting-styles.ts'

class SimpleGreeting extends LitElement {
  static styles = greetingStyles;
}
```

Code example 4.6: Importing external styles in a Lit component.

4.2.2.1 Dynamic styles & theming

Sometimes styling in components should change based on certain properties and actions. In other words, styling needs to be **dynamic**, and there are a few ways to accomplish this. Lit offers two directives, `styleMap` and `classMap` that allows the author to apply classes and styles in HTML templates [20]. Directives are functions that can be used in Lit to customize the way an expression renders. [Fig. 2.7](#) shows some different states of `tabs`, and

they are achieved using `classMap` by assigning classes to the component based on certain conditions, such as the values of the component's props:

Lit (TypeScript)

```
// tab.ts

import { classMap } from 'lit/directives/class-map.js';

render() {
  return (
    <div
      class=${classMap({
        'tab--active': this.active,
        'tab--closable': this.closable,
        'tab--disabled': this.disabled
      })}
    >
  )
}
```

Code example 4.7: Dynamic class assignment using `classMap`.

In [Code example 4.7](#) the `classMap` directive is applying the class `tab--active` if the property `this.active` (this referring to the `MOTab` class), evaluates to true. As a result, the tab is rendered differently to indicate that it is the currently selected and active tab. The same logic applies to the `disabled` and `closed` states that can be seen in [Fig. 2.7](#).

Theming can be achieved by utilizing CSS inheritance and CSS variables and custom properties together. It is possible to implement tree-based and per-instance theming by using CSS selectors to customize CSS custom properties [20]. As all CSS custom properties (`--*`) are inherited [6], they will propagate from an ancestor element to its descendants. This means that custom properties can be defined in the root of the HTML tree structure, and they will be available to all elements in the tree. A custom property called `--button-primary-color` can then be defined to have different colors for different themes. A change in the variable's value will be reflected to the component if it is used to define the component's style:

CSS

```
.button--primary {
  background-color: var(--button-primary-color);
}
```

Code example 4.8: Using custom properties to define component style.

4.2.3 Reactive properties

When receiving input, Lit components retain their current state in JavaScript class fields or properties. **Reactive properties** are those that, when modified, can cause the reactive update cycle. This cycle re-renders the component and, if desired, can read or write to the component's attributes. The `@property` decorator with a class field declaration declares a reactive property:

Lit (TypeScript)

```
@property()  
name = 'Default name';
```

The `@property` decorator can take an options object as an argument. These arguments include specifying the property's type (**type**), modifying its reactive behavior (**reflect** and **attribute**), or making it an internal state property (**state**). Leaving the arguments field empty (as in the example above) will have the same result as specifying all the default values for the options.

4.2.3.1 Public properties & internal state

The component's public API includes **public properties**. Public properties should generally be regarded as input, especially public reactive properties. A component shouldn't alter its own public attributes unless prompted by a user. A menu component, for instance, may include a public chosen property that the element's owner can initialize to a certain value, but that is changed by the component when the user picks an item. The component should emit an event in these situations to inform the component's owner that the chosen property has changed. [20]

Lit also includes support for **internal reactive state**. Reactive attributes that aren't covered by the component's API are referred to as internal reactive state. These attributes are often designated as protected or private in TypeScript and lack a matching attribute.

Lit (TypeScript)

```
@state() // equivalent to @property({ state: true })  
private _counter = 0;
```

Code example 4.9: Declaring an internal state property for a component in Lit.

The only difference between internal reactive state and public reactive properties is that internal reactive state doesn't have an attribute attached to it. The **hasChanged** method is the only option that can be specified for an internal reactive state.

4.2.3.2 Attributes

While properties can be used to pass data as input, accessing them requires JavaScript or a framework such as React. With attributes, it is possible to configure an elements

initial values directly from markup, which is the standard method in HTML. Lit achieves maximum compatibility by providing both an attribute and a property interface to supply initial data to components. [20]

HTML

```
<my-button content="Click me"></my-button>
```

Code example 4.10: Assigning an initial value to a property through attributes.

React (TypeScript)

```
<MyButton content={"Click me"}></MyButton>
```

Code example 4.11: Assigning an initial value directly to a property in React.

By default, Lit sets up an observed attribute corresponding to each public reactive property, and updates the property when the attribute changes. By default the created attribute is a lowercase copy of the property name, but it is configurable. Additionally, it is possible to disable automatic attribute creation with the `attribute` option. See [Code example 4.12](#) for syntax examples.

Lit (TypeScript)

```
// observed attribute name is "examplevalue"
@property({ type: Number })
exampleValue = 0;

// Observed attribute will be called my-name
@property({ attribute: 'my-name' })
myName = 'Aleksi';

// No observed attribute for this property
@property({ attribute: false })
myData = {};
```

Code example 4.12: Different configurations for Lit attribute handling.

Properties can also be configured to **reflect** their value to the corresponding attribute whenever it changes (see [Code example 4.13](#)). This is useful as the attributes can be accessed via CSS or DOM APIs such as `querySelector`. However, reflecting should be used sparingly, as attributes are generally used only for input, rather than being controlled by the component. Reflecting properties such as arrays or objects is not recommended as serializing large objects to the DOM might lead to poor performance. [20]

```
// value of property "value" will be reflected to the
→ corresponding attribute
@property({ reflect: true })
active = false;
```

Code example 4.13: Reflecting property values to attributes.

4.2.4 Rendering & templates

Templates should be added to each component to define what it should render. These templates can include expressions (e.g. `${this.name}`), which are placeholders for dynamic content. The `render()` method defines the template of a Lit component:

```
render() {
  return html`<p>Hello, ${this.name}!</p>`
}
```

This syntax is similar to that of React's JSX, as it can include conditional rendering, logic, and dynamic content in a simple syntax that compiles to HTML. Lit's `html` tag function can be used to write the template in HTML inside a tagged template literal, similarly to the `css` tag mentioned in [Section 4.2.2](#). The `render()` method should not change the component's state or cause any side effects, and it should only use the components props, to produce a deterministic result.

Lit templates can be composed from other templates, that can even be imported from other elements. This allows for the creation of components through **atomic composition**. In [Fig. 2.7](#), the `tab group` component contains imported templates of the `tab` and `tab panel` component inside its `render()` method.

RESULTS

5.1 Modes UI

In order to showcase the viability of a technology agnostic design system, a new component library, called Modes UI (see [Fig. 5.1](#)), was created along with a documentation site to demonstrate component functionality. Initially, Modes UI will serve as a Proof Of Concept (POC):

Proof of concept: Evidence, typically deriving from an experiment or pilot project, which demonstrates that a design concept, business proposal, etc. is feasible. [30]

But, as the component library matures, it will serve as a Minimum Viable Product (MVP):

Minimum viable product: An early, basic version of a product (typically a computer program or piece of technology) that meets the minimum necessary requirements for use but can be adapted and improved in the future, especially after customer feedback. [30]

The components will be implemented in order of importance, i.e., the most common, core components will be implemented first (and to an extent simplicity, e.g., data visualization will not be prioritized). Re-creating all the components of DSUI using web components is a massive undertaking, and as such this MVP will only include a subset of all DSUI components. As ease of migration and React compatibility are of utmost importance, the MVP will seek to demonstrate how migration and usage inside a React project functions in practice.

In general, a proof of concept is used to determine whether an idea is feasible or to ensure that it will work as intended [32]. In the context of this thesis, for example, a proof of concept would demonstrate whether a concept is feasible from a technology standpoint. More specifically, concrete examples of components being implemented using web components, and showing how they would function compared to, and together with, existing React counterparts.

As part of this thesis, I set out to create an MVP of a technology agnostic component library for Metso Outotec, that would first function as the POC before maturing into a product that could actually be adopted by users. The criteria for the MVP in this context would be:

- it contains a subset of the most important and frequently used components from the old technology specific library

- the components are rendered on a documentation site with multiple usage examples
- each component has test cases built for them, to allow automated testing
- the documentation site has exhaustive documentation on how to adopt and migrate to the new component library
- it has well documented and functional React support

This section will go through the implementation process of the new component library MVP, that goes by the name of Modes UI.

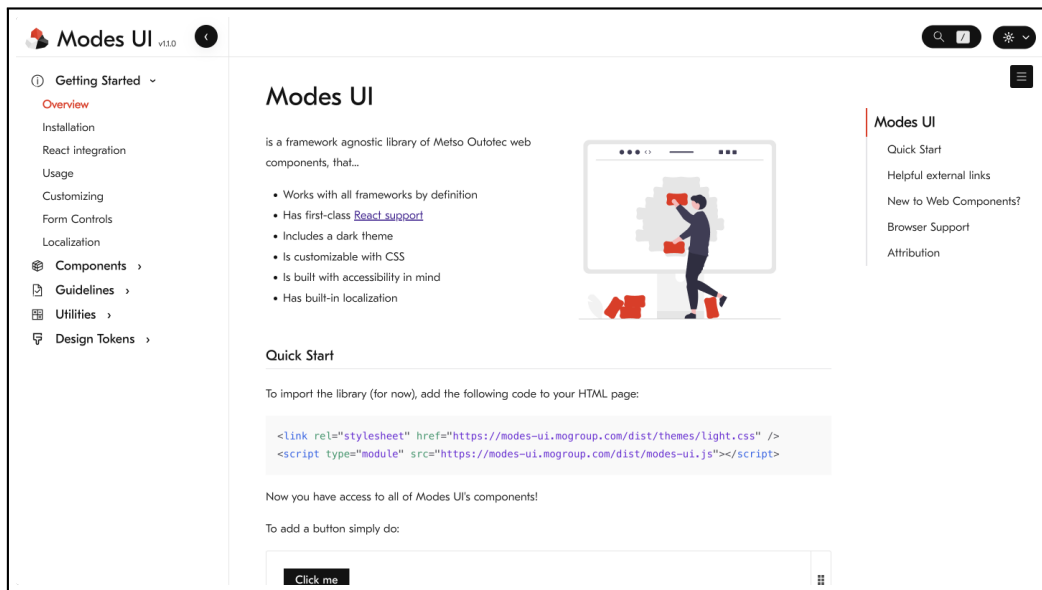


Figure 5.1: Modes UI documentation website, available at <https://modes-ui.mogroup.com>

After extensive testing and comparisons, Lit was chosen as the library to add an abstraction level above Web Components that enables more efficient creation and maintenance of the created components. A great open-source component library called Shoelace [38] was used as the base of the project. It builds further on top of Lit, and has numerous existing components that have been implemented using Web Component best practices. Using this open-source resource saves hundreds of hours, and some of the company's fiscal savings can be given back to the developer behind it through a GitHub sponsorship.

Modes UI attempts to improve in some key areas where DSUI is lacking. It offers more flexible, customizable components, with improved documentation that allows users to copy and paste exact component code to use in their applications. It limits the amount of dependencies for a more reliable, smaller bundle. In addition to this, it comes with first-class React support, so it can be used alongside DSUI if needed. It does not cover all the components in DSUI, but adds some additional ones. The next subsections will expand upon these differences and similarities.

5.1.1 Project dependencies

The Modes UI project uses numerous different open-source tools and libraries to handle different needs. It inherits many of these dependencies from Shoelace [38], which provides a good base for development without introducing any unnecessary dependencies. The most important **"dependencies"** and **"devDependencies"**, and their descriptions are listed in the tables below.

Modes UI contains relatively few production level dependencies to minimize bundle size and mitigate risk in conflicts. Only libraries that are essential to the core functionality of the project are included in **"dependencies"**. Components are created using Lit, and the `@lit-labs/react` package improves React support and compatibility. `@floating-ui/dom` is used across any component that has hidden content that is shown as a "floating" element on interaction. It simplifies the component implementation significantly while introducing minimal overhead in terms of performance and bundle size. The `color` package is not strictly necessary, but it is a convenient utility library that is minimal in size.

Dependency	Description
"lit"	Lit is a simple library for building fast, lightweight web components (see Section 4.2).
"@lit-labs/react"	React integration for Web Components and Reactive Controllers.
"@floating-ui/dom"	Floating UI is a tiny, low-level library for creating "floating" elements like tooltips, popovers, dropdowns, menus, and more.
"color"	JavaScript library for immutable color conversion and manipulation with support for CSS color strings.
"lit-flatpickr"	A Lit port of the popular "flatpickr" JavaScript library, a lightweight yet powerful datetime picker.

Table 5.1: Modes UI **"dependencies"** table.

Including **"lit-flatpickr"** is arguably unnecessary as it is only used in one component. However, the `DateTime Picker` component is an integral part of a design system, and ensuring it is feature-rich and stable is worth the additional dependency. Furthermore, most users will likely be using the `DateTime Picker`, so it will not be a redundant dependency in most cases.

The **"devDependencies"** table is not an exhaustive list of all the local development and testing dependencies, e.g., `eslint` has multiple different plugin dependencies to add pattern recognition for lit, ally, markdown, et cetera that are not included in the base `eslint` package. The **"devDependencies"** bring many readability and maintainability benefits (see [Section 2.1.1.1](#)), along with tools for documentation generation and testing.

Dependency	Description
"typescript"	TypeScript provides optional types to JavaScript that support tools for large-scale JavaScript applications for any environment.
"eslint"	ESLint is a tool for identifying and reporting on patterns found in ECMAScript/JavaScript code.
"esbuild"	An extremely fast JavaScript bundler and minifier.
"docsify-cli"	Docsify is a static document website generator.
"@web/test-runner"	A test runner for web applications. Part of the Modern Web project, that aims to provide developers with the guides and tools to build a modern web.
"@open-wc/testing"	Provides testing for Web Components.

Table 5.2: Modes UI "devDependencies" table.

5.1.2 DSUI comparison

Modes UI and **DSUI** both serve the same types of applications and users and ideally they would be close to identical in terms of components and features offered. However, due to time constraints and some technological differences between Web Components and React components, there is some variation between the two component libraries. This subsection will compare the two libraries, and the benefits of each.

5.1.2.1 Components

Initially the plan was to have a subset of components from DSUI replicated as closely as possible in Modes UI. Components and their properties should have consistent naming if possible to prevent confusion, and to ease the migration process. It became clear, however, that it made sense to allow for some deviation between the two libraries, as long as the components served the same purpose with (more or less) the same features, and branding was consistent across them.

Most of the DSUI components are also present in Modes UI, with almost identical style and functionality. [Table 5.3](#) shows a visual representation of the exact discrepancies between the two libraries. Import components that are still missing from Modes UI include all charting components, `Table`, `Header`, and to a lesser extent `InteractiveGrid`, `WizardProcessIndicator` and `Onboarding`. On the other hand, Modes UI introduces some new components that are not yet in DSUI. `Skeleton` will be added to DSUI in the near future, but it is already available in the new component library. `Accordion` and `Autocomplete` are fairly simple components whose behavior can be mimicked by combining multiple DSUI components and adding another layer of logic, but using the ready-made components guarantees good functionality and saves time in implementation.

More complex components, such as the `Datetime picker`, use a third party library as

Component	DSUI	Modes UI
Accordion	✗	✓
Autocomplete	✗	✓
Charts	✓	✗
Header	✓	✗
InteractiveGrid	✓	✗
Onboarding	✓	✗
Range	✗	✓
Skeleton	✗	✓
Table	✓	✗
WizardProcessIndicator	✓	✗

Table 5.3: Component differences between DSUI and Modes UI.

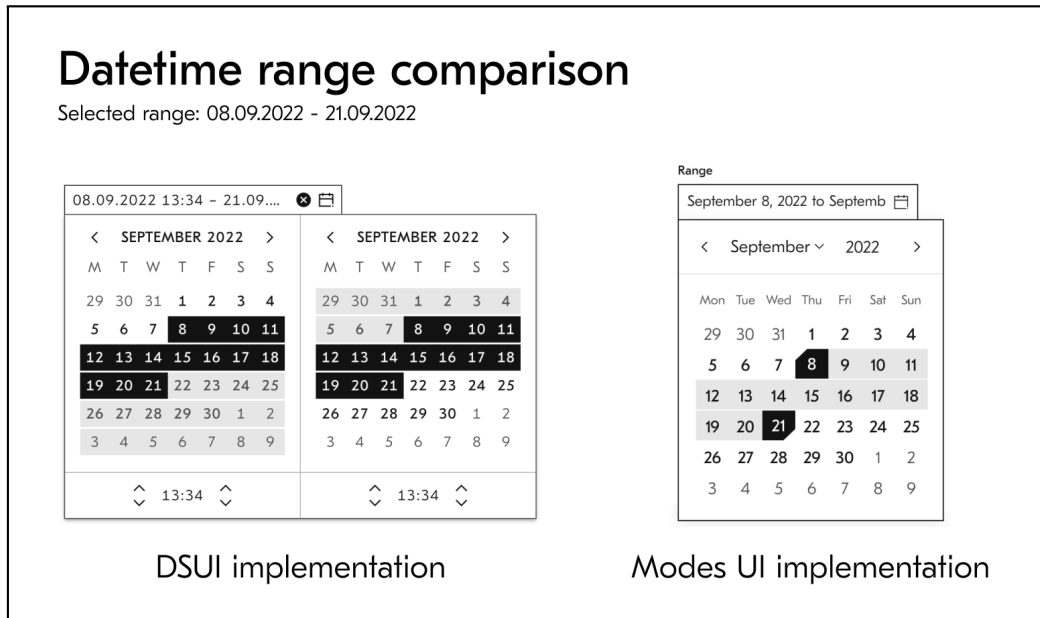


Figure 5.2: Differences between date time range implementations.

the base for both DSUI and Modes UI. The DSUI implementation is based on the `react-datetime` library, which obviously is not available outside React. The Modes UI implementation uses a Lit port of `flatpickr`, which is a general JavaScript library. Due to this, they have slight differences in offered features, and the visual style. Both implementations offer the essential features, and users can choose which one to use based on needs and preference. Having two slightly different versions introduces inconsistency

across the design system, which can lead to confusion. On the other hand, two different versions offer more options for the users, both feature and style wise. Time spent trying to copy the component one-to-one can instead be used on features or components that will bring more value.

Perhaps the biggest difference is in the range picker, where DSUI has two separate windows for the starting date and the ending date, while Modes UI has one window where the user can click twice to select both, and a visual cue that supports this workflow. This could be preferred in use cases where there is not much horizontal space (e.g., inside `dialogs`, `sidebars` or `drawers`).

5.1.2.2 Documentation

As mentioned in [Section 4.1.3](#), Modes UI uses an external library called `docsify` to generate its documentation. Compared to DSUI's Storybook implementation the websites performance is better, and it can offer additional documentation in-line with the component examples in a way that is highly customizable. Creating a documentation page from scratch is faster with Storybook however, and its tools that allow for interactive property value changes create additional value for users.

Syntax comparison

Small Medium Large

```

<mo-button size="small">Small</mo-button>
<mo-button size="medium">Medium</mo-button>
<mo-button size="large">Large</mo-button>

```

HTML REACT SOURCE ^ CODEPEN

Regular web component

Small Medium Large

```

import { M0Button } from 'modes-ui/dist/react';

const App = () => (
  <>
    <M0Button size="small">Small</M0Button>
    <M0Button size="medium">Medium</M0Button>
    <M0Button size="large">Large</M0Button>
  </>
);

```

HTML REACT SOURCE ^ CODEPEN

React-wrapped component

Figure 5.3: Source code is provided for both general usage and React specific usage.

A problem that has been re-occurring when users have used Storybook to find examples on how to use components, is that they are unable to easily find and copy the source code, and change it according to their needs. This is due to Storybook requiring the stories to be written in a specific fashion to actually be able to automatically show the exact source code, without hiding values behind `{...args}`. And even if the full source code is shown, Storybook code does not actually translate one-to-one to actual React code due to differences in internal state retention.

Due to this, Modes UI offers *exact* source code for both HTML and React usage, and even a CodePen link to a component playground for even easier demoing (see [Fig. 5.3](#)). This allows users to directly copy and paste real React code, and see immediate results. This is



Figure 5.4: Modes UI component documentation example.

easier for the users, and also reduces the workload of the design system developers as they do not have to address Storybook specific questions and issues. Using `docsify` to render the components in static HTML using markdown has the added benefit of being able to easily include additional documentation text for each example.

5.1.2.3 Development

There are differences and similarities in the development process as well. Evidently, the biggest difference is in developing components in Lit as opposed to React. Both Modes UI and DSUI use TypeScript, which is good for consistency. In addition to differences between React and Lit component creation that were discussed in [Section 4.2](#), differences in tools used also have an effect on the development process.

As React has garnered immense popularity since its release in 2013, many newer tools and frameworks have taken inspiration from it. Lit is not an exception to this, its core concepts and features are similar to those of React, although there are some key differences. According to a popular JavaScript framework benchmarking tool, Lit's html templating is around eight to ten percent faster than React's virtual DOM. Lit's minified bundle size is also smaller, at 5 kB compared to React's 40 kB [19]. These numbers indicate that Lit brings value compared to React in more ways than simply being technology agnostic.

Lit's templates (see [Section 3.2.3](#)) and React's JSX templating are incredibly similar, with a small syntactical difference in how multiple elements are grouped. React uses fragments, while Lit uses tagged template literals as previously mentioned. In terms of component creation, class components are, again, nearly identical. A bigger difference comes with states and properties, Lit's [Reactive properties](#) are a combination of React's props and hooks (e.g., `useState`) in terms of functionality, but using them takes some getting used to. The lifestyle cycle of both Lit and React is also essentially the same with different

naming conventions. All things considered, the learning curve when switching from React to Lit or vice-versa is rather shallow due to the amount of similarities between the two.

Differing choices in internal development tools such as bundlers and testing frameworks also introduce some differing practices in development. DSUI uses an older but extremely popular bundler in **webpack**, that is slower than Modes UI's **esbuild**. In Modes UI the library distributable is created in the **dist/** folder, whereas in DSUI it is in the **build/** folder. This naming convention difference between the bundlers causes **import** statements to differ slightly. Unit testing is done using Jest in DSUI, but as Jest is not available to Web Components (see [Section 4.1.1](#)), Modes UI uses a similar tool provided by Open Web Components (see [Table 5.2](#)).

Another noticeable difference is in performance when developing, DSUI's development environment has a long start-up time due to Storybook and webpack slowing it down. Modes UI has near instant start-up, and its hot-reload is also faster than that of DSUI's. Having to wait multiple seconds to see a change in code reflected in the rendered component may not seem like much, but it adds up and causes frustration for developers.

5.1.3 Providing React support

As React is widely adopted inside Metso Outotec, the new component library must ensure that it supports React usage. Modes UI is based on Web Components, and thus is framework agnostic by nature. Many frameworks, such as Angular and Svelte, work perfectly well with Web Components, and have full support for them. However, React in particular does not yet have complete Web Component support (as of September 2022). There is an **@experimental** release that **does** have full Custom Elements (and therefore, Web Components) support, but most users are using the **@stable** release. It may be introduced in an upcoming **@stable** release "React 19" in the near future, but at the time of writing Web Components need some additional workarounds to allow full usage inside React.

The issues are in particular with **data** and **event** handling. As discussed in [Section 4.2.3.2](#), Lit components use attributes to add user data to a component. This works in React as well, however, it only works for primitive data. When passing down rich data, such as arrays or objects, React will not properly apply these values to the component properties and will instead stringify them (e.g., **some-attr="object Object"**) [7]. The experimental React release will use a run-time heuristic to determine whether to pass the props as properties or attributes, which fixes this issue [7].

To improve React support, the developers of Lit offer an additional utility library. This library offers a utility function, called **createComponent** that can create React wrapped versions of components. In addition to listening for events sent by the custom element, the wrapper appropriately provides React props to properties allowed by the custom element [20]. Wrapping React components should be a part of the **build** step, so it is executed each time the library is built.

[Code example 5.1](#) is a shortened version of the script that is run on **build** to create new React wrapped components, that are written to a directory located in **'modes-ui/dist/react'**. These components can then be imported from that directory just as normal React components, and the syntax for their usage is also the same (see [Fig. 5.3](#)).

```

const components = getAllComponents(metadata);

components.map(component => {
  const componentFile = path.join(componentDir, 'index.ts');
  const importPath = component.modulePath.replace(/^src\/\//,
    => '').replace(/\.ts$/, '');
  const events = (component.events || []).map(event =>
    => `${event.reactName}: '${event.name}'`).join(',\n');

  const source = `
    import * as React from 'react';
    import { createComponent } from '@lit-labs/react';
    import Component from '../../${importPath}';

    export default createComponent(
      React,
      '${component.tagName}',
      Component,
      {
        ${events}
      }
    );
  `;
  // write 'source' to a new file to create new component
  fs.writeFileSync(componentFile, source, 'utf8');
});

```

Code example 5.1: Dynamically creating React wrapped components.

5.1.3.1 Event handling

In terms of **event handling**, React's synthetic events cannot directly listen to DOM events without a workaround, but this will also be fixed once the **@experimental** Custom Element support is brought to the main release of React [7]. There are a couple options to implement a band-aid fix to this issue. One way is manually creating new versions of component for React where using the **useRef** and e.g., **useLayoutEffect** React hooks the component will add the necessary event listeners on initial component render. However, this is cumbersome and especially for a larger component library simply not feasible.

A better way of ensuring that event handling works properly in react is using **CustomEvent**, which is a browser standard way of creating customized events. Authors may want to e.g., emit a different custom event for when a menu is opened, and when it is closed. These custom events can then be supplied to the **createComponent** function (see [Code example 5.1](#)) to allow for their usage inside React.

In [Code example 5.2](#), the custom event **onMoInput** returns a reference to the component, which contains data about the component's attributes (which are generated from the

```
import { useState } from 'react';
import { MOInput } from '@mogroup/modes-ui/dist/react';
import type MOInputElement from
  ↳ '@mogroup/modes-ui/dist/components/input/input';

function MyComponent() {
  const [value, setValue] = useState('');

  return <MOInput value={value} onMoInput={event =>
    ↳ setValue((event.target as MOInputElement).value)} />;
}

export default MyComponent;
```

Code example 5.2: Event handling in React using custom events.

component properties, see [Section 4.2.3.2](#)). This allows React users to easily access the components internal state and proceed to use the values in their own code. If using TypeScript, it is important to note that `event.target` needs to be either typed as `any`, or strongly typed as `MOInputElement` (as in the example above).

5.1.3.2 Form submission

As discussed in [Section 3.2.2](#), Lit components make use of the shadow DOM to encapsulate markup, styles, and behavior. One caveat of this approach is that native `<form>` elements do not recognize form controls located inside a shadow root. This problem is not unique to React usage, but the solution will work with React as well. This is solved inside Modes UI by using the `formdata` event, which is supported by all modern browsers. Modes UI's custom form controls will automatically append the forms values to the `FormData` object on form submission. This method allows users to create and handle forms as they normally would.

[Code example 5.3](#) shows a simple example of the standard `<form>` element used in React with Modes UI components inside it. It exemplifies that Modes UI components can be used in forms just as native elements would be, due to the additional `FormData` utility. The `pattern` property in `MOInput` allows for pattern checking on form submission in `RegExp`. The `required` property can be set to true by simply defining it in the component, just as in React. These properties will be checked on form submission to deduce if the submission is valid or if an error message should be thrown.

```
import { MOButton, MOInput } from '@mogroup/modes-ui/dist/react';

const App = () => {
  function handleSubmit(event) {
    event.preventDefault();
    alert('All fields are valid!');
  }

  return (
    <form onSubmit={handleSubmit}>
      <MOInput name="letters" required label="Letters"
        pattern="[A-Za-z]+" />
      <br />
      <MOButton type="submit" variant="primary">
        Submit
      </MOButton>
    </form>
  );
};
```

Code example 5.3: Form submission in React using Modes UI components.

5.2 Users

Modes UI's initial MVP was published on the company's internal npm registry on the 3rd of June. Since then, it has accrued a small user base that has used it to create small POC applications on non-React frameworks, or used it together with DSUI on React applications. Modes UI offers some additional components and utility functions that can be useful for those that are already employing DSUI in their projects, and the Web Component versions of some components are different enough to serve certain applications better. This section will review the results of user surveys that focus on the design system and technology agnosticism. Some Key Performance Indicators (KPIs) will also be analyzed to find how Modes UI has done so far. Lastly, a short case study will examine Modes UI being used by a product team inside the Vue framework for a POC.

5.2.1 Survey results

Inside a large corporation there are a large amount of people and teams that could contribute to a survey and offer useful data that can be used to improve the design system. However, successfully surveying this large potential user base proved to be rather difficult, due to lacking communication channels and the lack of interest/time from the teams to answer the survey. An initial survey conducted in July, using Microsoft Forms, received 16 responses from people working in ten different product teams, using eight different software products.

These users were surveyed anonymously, and the questions ranged from objective questions

such as which front-end framework(s) they were using, to more subjective ones, such as what improvements they would like to see in the design system. The main purpose of the survey was to assess how many users and teams would currently benefit from a technology agnostic implementation of the component library, and how it could improve the existing implementation. The surveyees were mainly software developers, with a couple product owners and UX designers in the mix as well.

The sample size is limited and not statistically significant, but the results do still offer some valuable insight in to the user base of the design system. 56% of the respondents answered that they are currently using React as their front-end framework of choice, while 19% reported that they are using something else. It is unclear from the responses whether those 19% could use Web Components instead, but it is clear that not every one uses React. Surveyees were asked to rate on a scale of one to ten, how well they know the design system. The average rating was 6.75, while the mean was 7. By definition 5 would be average, but considering the human psychology, 6.75 is not a great score, and not necessarily above average. This indicates there is improvement to be made on spreading knowledge about **what** the design system is, **how** to use it, and **why** they should use it. All in all, the respondents gave Modes a rating of 3.6 out of 5, which is good but there is definitely room for improvement.

For the free-form data, there were interesting points that came up more than once. Developers gave feedback on the components, and their pain points were about performance, code examples (see [Section 5.1.2.2](#)), and them being hard to use, especially when they had external dependencies. A separation of logic for the **Table** component brought up a key talking point: should the design system component library provide the logic for tables, or just the visual styling tokens and guidelines users can utilize to create their own? As a table is a complex component with varying feature requirements for different users, it is difficult to create a one-size-fits-all implementation. These concerns were taken in to account when creating the new component library, and as a result Modes UI should be more user-friendly than DSUI due to improved documentation and reduced dependencies.

Product owners and UX designers gave more broad feedback for the design system itself. As Metso Outotec's digital products are used in an industrial context, accessibility and simple color schemes have been the building blocks of the design system. However, responses asked for additional colors to improve the UX, and more compact elements. Due to accessibility concerns some elements are visually big which makes creating compact views difficult. Allowing for smaller sizes would give users more options, but also shift the responsibility of ensuring accessibility to the users. At the same time, there was also a request for more accessibility guidelines, which would support this paradigm. There was some need for additional documentation for the design tokens, e.g., colors, fonts, sizing, as some users only used the design system for those and created their implementations without the component library.

5.2.2 Key performance indicators

The main KPIs that can be tracked for a component library are downloads and unique users across different versions of the library. For Modes UI, the library has been downloaded a total of 769 times by 16 unique users since its release in the beginning of June (as of September 1st). These users have not provided much additional feedback on how they are

actually using the components, or if they are still actively using them. Regardless, the numbers are impressive and serve as proof that there is a use case to the new component library and that it is providing value to users. Of the 16 unique users, only four have updated to the latest version (published on August 8th). This shows that users are not very active in checking for newer versions, but this was to be expected. One user of DSUI said that in the past there were often breaking changes that required heavy refactoring, which was laborious and it made them hesitant to upgrade to the newest major version.

The same applies to DSUI, which has 14k+ downloads and 115 unique users since its inception over three years ago. The latest releases of DSUI also get around one fourth of the total unique users, around 30 users have updated to the latest stable version. This is due to some of those users no longer using the library, or not wanting to upgrade to a major release and deal with breaking changes. Nevertheless, over thirty active users is an impressive amount that serves as a testament to the value of the design system as a whole. Some of these users may also use Modes UI in the same projects, and hopefully the technology agnostic component library will be able to reach new unique users who are not using React.

The design system documentation at zeroheight allows the collection of analytics data through Google Analytics. The data from zeroheight shows that the design system documentation is used also by those who are not direct users of the component libraries. The site has 293 unique visitors since the start of the year, and they spend around 5 minutes per visit on the page. Each user has visited the site on average 4 separate times, and 77% of visitors return after their first visit. The analytical data gives a good understanding of the design systems global nature as well, as evident from [Table 5.4](#).

Continent	Users	New users
Europe	128	107
Asia	77	73
Americas	60	60
Oceania	27	26
Africa	8	7

Table 5.4: Modes is regularly accessed from all over the globe.

In terms of localization, **en-us** is by far the most common language preference amongst visitors, with 223 out of the 293 users (around 76%) preferring it. **en-gb** is next with 26 users, and **fi-fi** is third with 12 users. Approximately 60 percent of sessions are from Finland, but only approximately four to five percent of users actually use Finnish as their preferred locale. The option for translations and localization are important to provide, but as seen by these numbers most users will prefer to use English anyway. Thus it makes little sense to implement ready-made translations for tons of different languages, and instead users should be able to create their own translation files to be used in the component library.

5.2.3 Case study: Intelligent platforms

Inside Metso Outotec there is continuous testing and creation of new digital projects, products, POCs and MVPs by different product teams. Some of these new digital products want to use the Metso Outotec design system, but may want to choose a framework different from React. One such case is a POC created by the Intelligent instruments team, who adopted Modes UI in a Vue-based project. They create platforms for intelligent instruments to be used in automation. They mainly provide tools, instead of fully-fledged products. The POC was created by a single developer, and it has since been successfully used in demos to showcase hardware functionality.

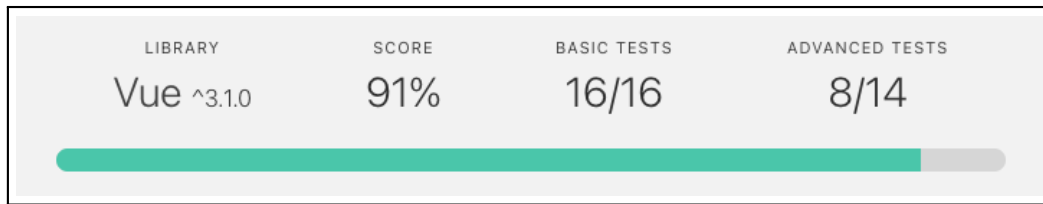


Figure 5.5: Vue supports almost all Web Component features. [7]

Their decision to choose Vue as the framework was due to previous familiarity, and interest in finding out how well the newest release of Vue would fit their needs. The developer said that for this POC in particular he would not have used DSUI even if he had used React as it would have been considerably heavier and slower to integrate in to the project. Convenience was a major upside of Modes UI, as the developer was able to include it in the project with a few lines of code and he did not have to deal with local installation and back-end configuration to get started. Additionally, the whole web application would run on a Raspberry Pi, which means that minimizing bundle size is critical.

The developer reported that there were minimal issues with Modes UI usage, although importing icons locally to the project required some manual folder re-structuring. They also had some issues with accessing the private npm registry, but that is more of a DevOps issue. As it was a fairly simple POC, only a handful of different Modes UI components were used. These include basic components such as `mo-input`, `mo-button`, `mo-checkbox`, `mo-dialog` and `mo-card`. The developer said he was missing the header component, that is available in DSUI. On the other hand, he found Modes UI's `mo-details` useful and said it was a good addition that DSUI does not include.

In conclusion, Modes UI was able to offer additional value by enabling the user to use Vue, since they were most comfortable using it. The developer would have rather chosen to forego the component library if DSUI was the only option, but chose to try out Modes UI as it was lighter and much faster to start using.

CONCLUSION

A **design system** is a set of guidelines for managing design at scale by minimizing duplication and establishing a common visual language and style across many pages and channels. Establishing a design system in a large company has innate value, that only increases as time passes. This value is generated mostly through improving working efficiency for both developers and designers, and can be measured quantitatively to an extent. Other notable value generators are stronger branding and being able to ensure accessibility from top-down, which lead to a better user experience. A design system's value directly correlates to the amount of users it has. Therefore it is important to ensure maximum compatibility and that potential users are aware of the design system.

Most design systems have component libraries, which are collections of re-usable components that can be used to create consistent and efficient user interfaces for digital products. These component libraries are often built using a specific technology or framework, which can limit their compatibility with other technologies. In the case of Metso Outotec and many other design systems, this technology is React. This means that a product team can only fully utilize the design system if they are also using React as their front-end framework. This restricts users in to choosing between the component library and an alternative framework. The way to solve this is to create a **technology agnostic** component library, which is compatible with all the different tools and technologies users may want to employ.

In recent years, there has been tremendous progress in the industry in the creation of technology agnostic design systems. Google, Microsoft and Adobe have all created a new component library, using **Web Components**. Web Components are a relatively new browser standard that is supported by all modern browsers. They allow for the creation of custom elements that include both styling (CSS) and logic (JavaScript), without requiring an external framework or library to function in the browser. Creating Web Components from scratch is tedious, and a library or framework will improve the development process. Google's **Lit** and Ionic's **Stencil** are the most popular Web Component libraries. Metso Outotec's new technology agnostic component library is built using Lit.

A component library is not trivial to build from scratch. Luckily there are multiple open source resources that help with the initialization of a component library, and some even offer a base set of components that authors can customize. Package management and version control are important aspects of component library maintenance, and should be planned out well. Testing is a critical, yet often overlooked facet in development. Components should have tests for individual features and ensure the component functionality does not break between releases. These tests should be run whenever a new feature is added. Testing can be split into unit testing and end-to-end testing. Unit testing is the classic way to test individual components, but end-to-end testing has gained prominence and is better at simulating a real-world scenario. Documentation and distribution must also be a priority, as they are important for user acquisition and retention.

Lit is a simple and lightweight library for the creation of high quality and performant web

components. It is similar to React and other modern frameworks in terms of development workflow, which makes it easier to pick up for those with experience in that field. Lit has many valuable features that can be utilized to guarantee the components are not only technology agnostic, but also high in quality. Creating components that are composable, compatible and accessible makes the component library maintenance process easier.

Modes UI is now a fully functional component library that has been installed by almost 20 users in a couple months. It is still missing some key components, and wider adoption may bring up issues, but it is already providing value for the users. It serves as proof that a technology agnostic component library is feasible, and will be used alongside **DSUI** for the time being. In the future, it will ideally offer all the components that DSUI does. Having two component libraries is not ideal, but for the moment it is necessary. Full migration from using both DSUI and Modes UI to only using Modes UI is soon possible, but will realistically take a long time to materialize, if at all. Regardless, with Modes UI more product teams can take full advantage of the design system, and have more freedom when choosing frameworks for their future projects.

REFERENCES

- [1] Accessed: 12-08-2022. 2022. URL: <https://stenciljs.com/>.
- [2] P. Brophy et al. “Web accessibility”. In: *Library trends* 55.4 (2007), pp. 950–972.
- [3] *Browser market share statistics*. Accessed: 17-03-2022. 2021. URL: <https://gs.statcounter.com/browser-market-share>.
- [4] *caniuse*. Accessed: 17-03-2022. 2022. URL: <https://caniuse.com/custom-elementsv1>.
- [5] *Carbon Design System*. Accessed: 08-08-2022. 2022. URL: <https://carbondesigntsystem.com/>.
- [6] *CSS Inheritance*. Accessed: 14-08-2022. 2022. URL: <https://developer.mozilla.org/en-US/docs/Web/CSS/inheritance>.
- [7] *Custom Elements Everywhere*. Accessed: 23-08-2022. 2022. URL: <https://custom-elements-everywhere.com/>.
- [8] D. Denicola et al. “Custom elements”. In: *World Wide Web Consortium, Working Draft WD-custom-elements-20161002* (2016).
- [9] *docsify*. Accessed: 11-07-2022. 2022. URL: <https://docsify.js.org/#/?id=docsify>.
- [10] *E2E testing best practices*. Accessed: 15-08-2022. 2022. URL: <https://www.pagerduty.com/blog/end-to-end-e2e-testing-best-practices/>.
- [11] *esbuild*. Accessed: 15-07-2022. 2022. URL: <https://esbuild.github.io/>.
- [12] *ESLint*. Accessed: 11-08-2022. 2022. URL: <https://eslint.org/>.
- [13] B. Farrell. *Web Components in Action*. Simon and Schuster, 2019.
- [14] T. Fessenden. *Design systems 101*. Accessed: 07-03-2022. 2011. URL: <https://www.nngroup.com/articles/design-systems-101/>.
- [15] B. Frost. *Atomic design*. Brad Frost Pittsburgh, 2016.
- [16] *Human Interface Guidelines*. Accessed: 08-08-2022. 2022. URL: <https://developer.apple.com/design/human-interface-guidelines/guidelines/overview/>.
- [17] *InVision guide: Benchmarking Your Design System*. Accessed: 06-04-2022. 2020. URL: <https://www.invisionapp.com/lp/design-system-benchmarking-guide/>.
- [18] *JavaScript testing best practices*. Accessed: 15-08-2022. 2022. URL: <https://blog.logrocket.com/javascript-testing-best-practices/>.

- [19] *JS framework benchmark*. Accessed: 24-08-2022. 2022. URL: <https://github.com/krausest/js-framework-benchmark>.
- [20] *Lit.dev*. Accessed: 04-07-2022. 2022. URL: <https://lit.dev/>.
- [21] *Managing technology agnostic design systems*. Accessed: 25-08-2022. 2022. URL: <https://bradfrost.com/blog/post/managing-technology-agnostic-design-systems/>.
- [22] *Material Design*. Accessed: 08-08-2022. 2022. URL: <https://material.io/design>.
- [23] *MDN Web Docs - Introduction to the DOM*. Accessed: 12-07-2022. 2022. URL: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction.
- [24] *MDN Web Docs - Template literals*. Accessed: 11-07-2022. 2022. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals.
- [25] *MDN Web Docs - Using shadow DOM*. Accessed: 12-07-2022. 2022. URL: https://developer.mozilla.org/en-US/docs/Web/Web_Components/Using_shadow_DOM.
- [26] *MDN Web Docs - Web Components*. Accessed: 05-04-2022. 2022. URL: https://developer.mozilla.org/en-US/docs/Web/Web_Components.
- [27] *npm Docs*. Accessed: 01-08-2022. 2022. URL: <https://docs.npmjs.com/>.
- [28] *npm trends*. Accessed: 15-07-2022. 2022. URL: <https://npmtrends.com/>.
- [29] *Open Web Components - testing*. Accessed: 22-07-2022. 2022. URL: <https://open-wc.org/docs/testing/testing-package/>.
- [30] Oxford University Press. *Oxford dictionary*. Accessed: 29-06-2022. 2021. URL: <https://www.lexico.com/en/>.
- [31] C. Pemberton. *8 Top Findings in Gartner CMO Spend Survey 2018-19*. Accessed: 07-04-2022. 2018. URL: <https://www.gartner.com/en/marketing/insights/articles/8-top-findings-in-gartner-cmo-spend-survey-2018-19/>.
- [32] M. K. Pratt. *What is proof of concept (POC)?* Accessed: 11-04-2022. 2020. URL: <https://www.techtarget.com/searchcio/definition/proof-of-concept-POC/>.
- [33] *Prettier*. Accessed: 11-08-2022. 2022. URL: <https://prettier.io/>.
- [34] T. Rangnau et al. "Continuous security testing: A case study on integrating dynamic security testing tools in ci/cd pipelines". In: *2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC)*. IEEE. 2020, pp. 145–154.

- [35] B. Ray. *How much is a design system worth?* Accessed: 07-04-2022. 2018. URL: <https://uxdesign.cc/how-much-is-a-design-system-worth-d72e2eddf76/>.
- [36] M. Rieger et al. “Insights into system-wide code duplication”. In: *11th Working Conference on Reverse Engineering*. IEEE. 2004, pp. 100–109.
- [37] *Semantic Versioning*. Accessed: 22-08-2022. 2022. URL: <https://semver.org/>.
- [38] *Shoelace - a forward thinking library of Web Components*. Accessed: 04-06-2022. 2022. URL: <https://shoelace.style/>.
- [39] C. Slack. *Measuring the value of design systems*. Accessed: 06-04-2022. 2019. URL: <https://www.figma.com/blog/measuring-the-value-of-design-systems/>.
- [40] *StackOverflow user survey*. Accessed: 04-03-2022. 2021. URL: <https://insights.stackoverflow.com/survey/2021#section-most-popular-technologies-web-frameworks>.
- [41] *The state of front-end frameworks*. Accessed: 25-08-2022. 2022. URL: <https://2021.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- [42] *TypeScript: Documentation - Decorators*. Accessed: 18-08-2022. 2022. URL: <https://www.typescriptlang.org/docs/handbook/decorators.html>.
- [43] *W3C Accessibility Guidelines*. Accessed: 11-08-2022. 2022. URL: <https://www.w3.org/standards/webdesign/accessibility>.
- [44] *Web Component libraries*. Accessed: 05-07-2022. 2022. URL: <https://www.webcomponents.org/libraries>.
- [45] *What Is a Component Library and Why Should You Use One for UI Development?* Accessed: 10-08-2022. 2022. URL: <https://www.uxpin.com/studio/blog/ui-component-library/>.
- [46] *What Is a Linter? Here’s a Definition and Quick-Start Guide*. Accessed: 10-08-2022. 2022. URL: <https://www.testim.io/blog/what-is-a-linter-heres-a-definition-and-quick-start-guide/>.
- [47] *What is DevOps?* Accessed: 15-08-2022. 2022. URL: <https://aws.amazon.com/devops/what-is-devops/>.
- [48] E. Wittern et al. “A Look at the Dynamics of the JavaScript Package Ecosystem”. In: *2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR)*. 2016, pp. 351–361.